

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY

Jeffrey J. McConnell

Analysis *of* Algorithms

Second Edition
An Active Learning Approach

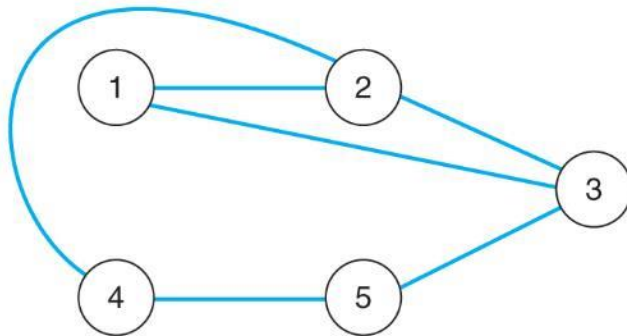


Chapter 8

Graph Algorithms

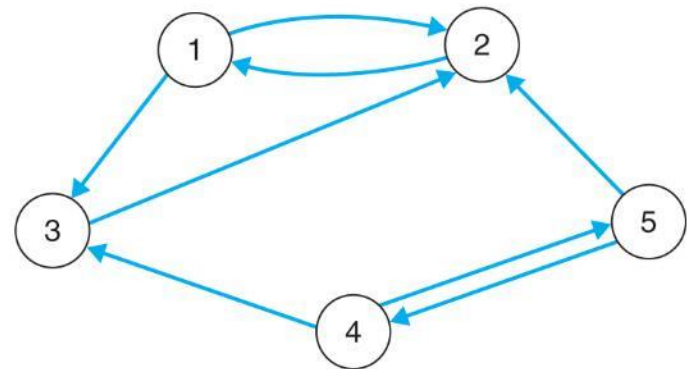
Graph Types

- A directed graph edges' allow travel in one direction
- An undirected graph edges' allow travel in either direction



■ FIGURE 8.1A

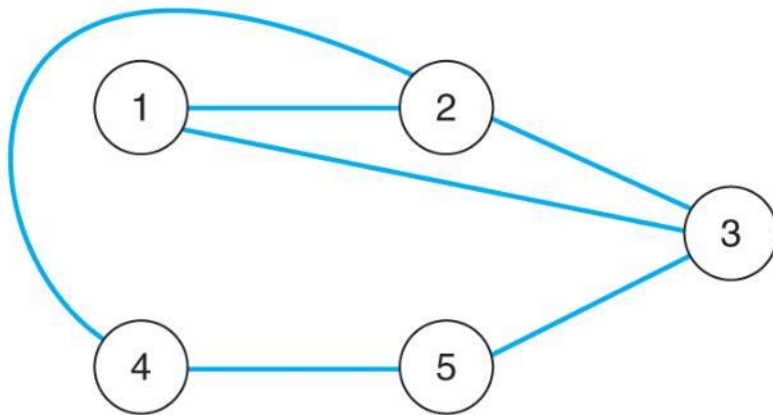
The graph $G = (\{1, 2, 3, 4, 5\}, \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}\})$



■ FIGURE 8.1B

The directed graph $G = (\{1, 2, 3, 4, 5\}, \{(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5, 4)\})$

Adjacency Matrix Example 1



■ **FIGURE 8.1A**

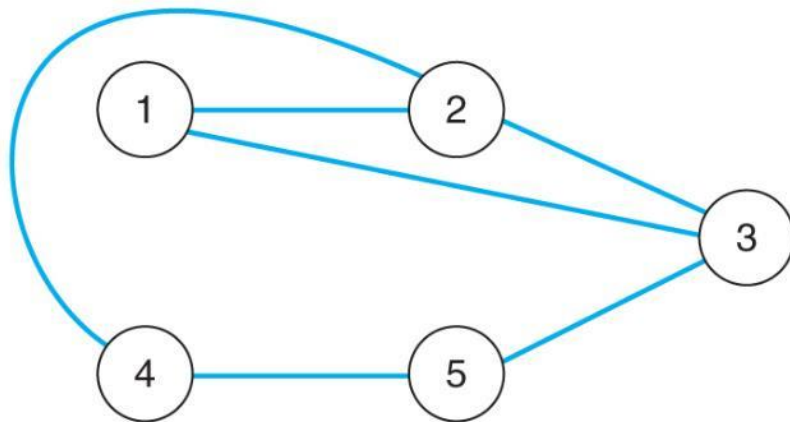
The graph $G = (\{1, 2, 3, 4, 5\}, \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}\})$

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

■ **FIGURE 8.2A**

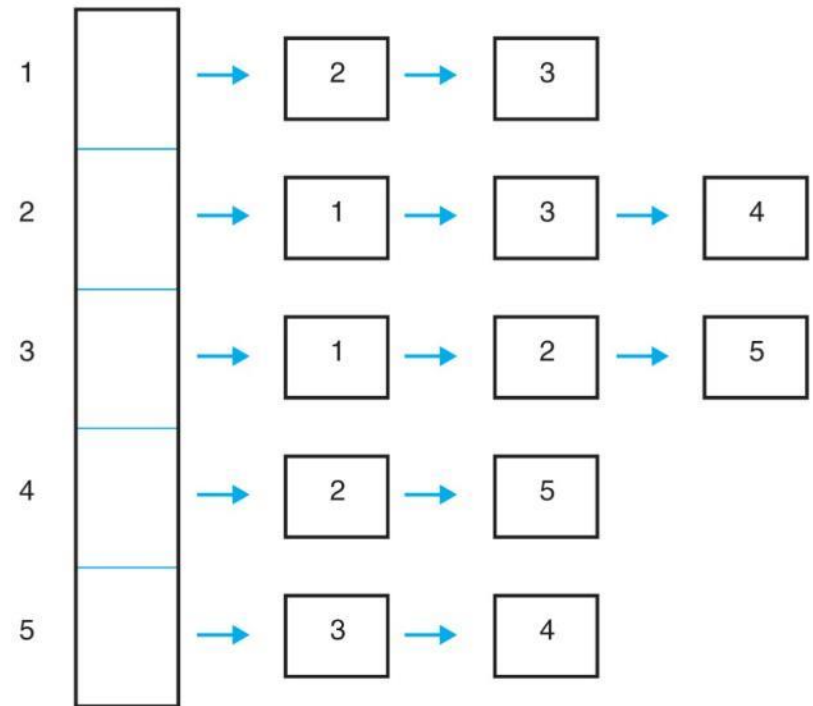
The adjacency matrix for the graph in Fig. 8.1(a)

Adjacency List Example 1



■ **FIGURE 8.1A**

The graph $G = (\{1, 2, 3, 4, 5\}, \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}\})$



■ **FIGURE 8.3A**

The adjacency list for the graph in Fig. 8.1(a)

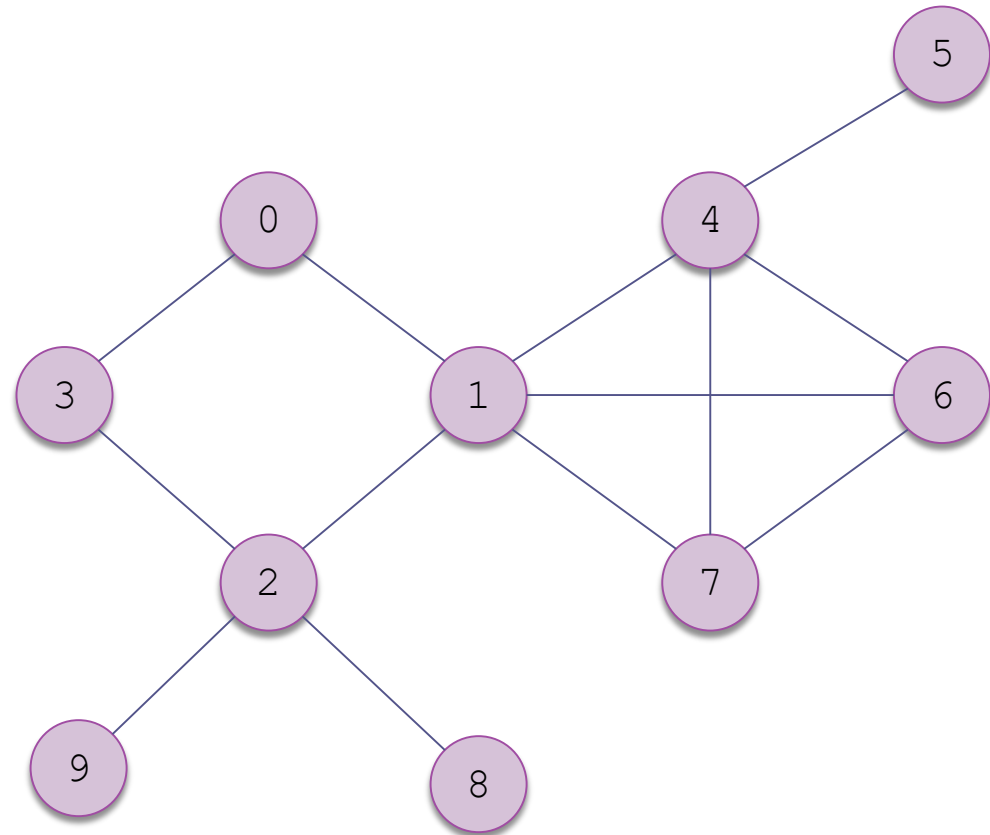
Traversals of Graphs

- Most graph algorithms involve visiting each vertex in a systematic order
- The two most common traversal algorithms are the *breadth-first search* and the *depth-first search*

Breadth-First Search

- In a breadth-first search,
 - ▣ visit the start node first,
 - ▣ then all nodes that are adjacent to it,
 - ▣ then all nodes that can be reached by a path from the start node containing two edges,
 - ▣ then all nodes that can be reached by a path from the start node containing three edges,
 - ▣ and so on
- We must visit all nodes for which the shortest path from the start node is length k before we visit any node for which the shortest path from the start node is length $k+1$
- There is no special start vertex— we arbitrarily choose the vertex with label 0

Example of a Breadth-First Search



0 unvisited

0 visited

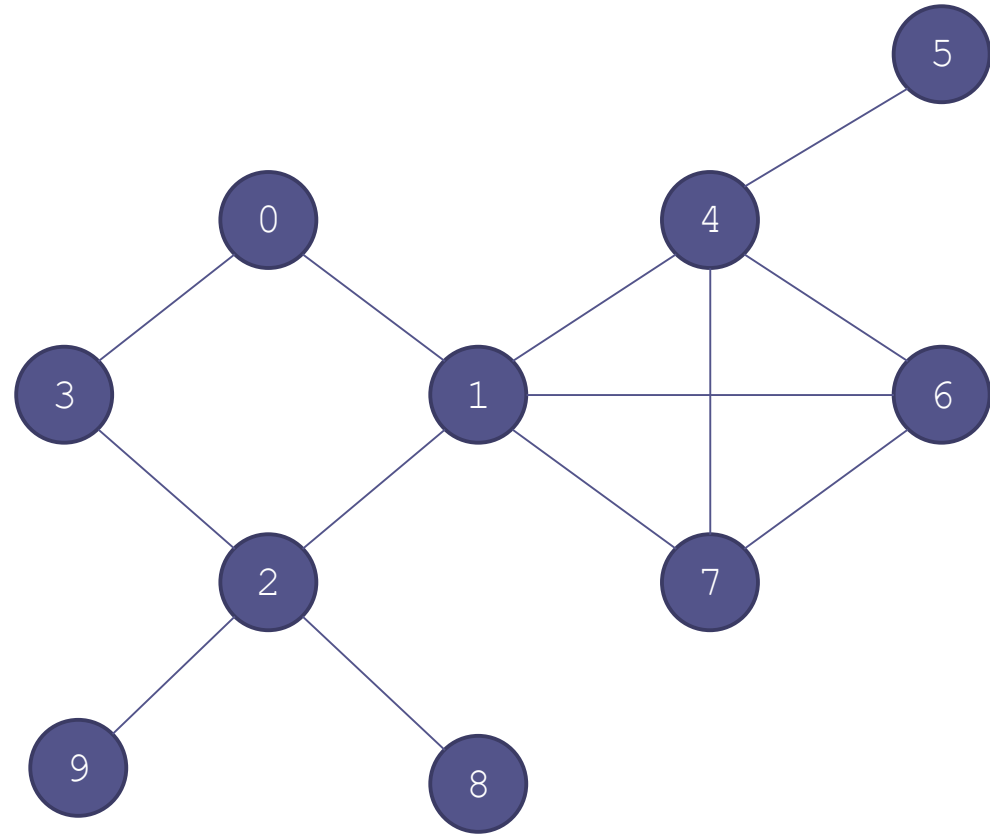
0 identified

Example of a Breadth-First Search (cont.)

The queue is
empty; all vertices
have been visited

Queue:
empty

Visit sequence:
0, 1, 3, 2, 4, 6, 7, 8, 9, 5



0 unvisited

0 visited

0 identified

Algorithm for Breadth-First Search

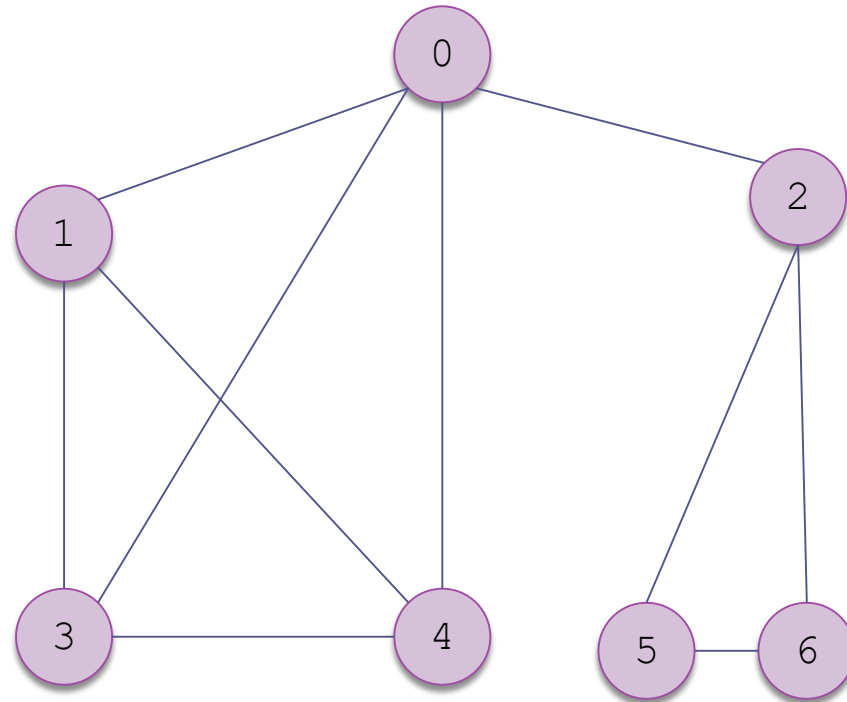
Algorithm for Breadth-First Search

1. Take an arbitrary start vertex, mark it identified (color it light blue), and place it in a queue.
2. **while** the queue is not empty
3. Take a vertex, u , out of the queue and visit u .
4. **for** all vertices, v , adjacent to this vertex, u
5. **if** v has not been identified or visited
6. Mark it identified (color it light blue).
7. Insert vertex v into the queue.
8. We are now finished visiting u (color it dark blue).

Depth-First Search

- In a depth-first search,
 - ▣ start at a vertex,
 - ▣ visit it,
 - ▣ choose one adjacent vertex to visit;
 - ▣ then, choose a vertex adjacent to that vertex to visit,
 - ▣ and so on until you go no further;
 - ▣ then back up and see whether a new vertex can be found

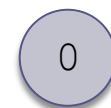
Example of a Depth-First Search



unvisited



visited



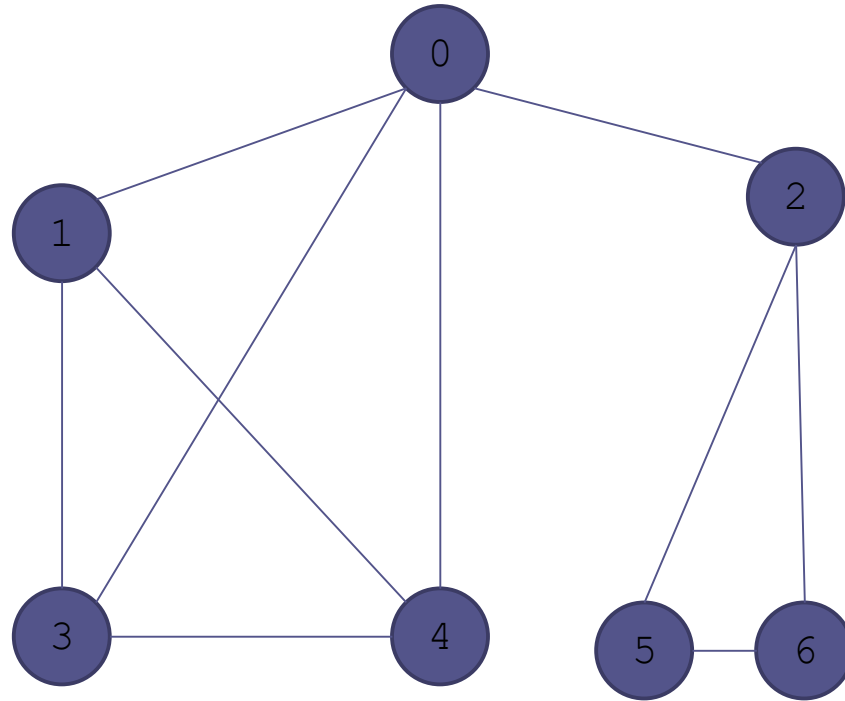
being visited

Example of a Depth-First Search

(cont.)

Mark 0 as visited

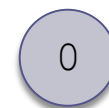
Discovery (Visit) order:
0, 1, 3, 4, 2, 5, 6, 0



unvisited



visited



being visited

Algorithm for Depth-First Search

Algorithm for Depth-First Search

1. Mark the current vertex, u , visited (color it light blue), and enter it in the discovery order list
2. for each vertex, v , adjacent to the current vertex, u
3. if v has not been visited
4. Set parent of v to u .
5. Recursively apply this algorithm starting at v .
6. Mark u finished (color it dark blue) and enter u into the finish order list.

Performance Analysis of Depth-First Search

- The loop at Step 2 is executed $|E_v|$ times
- The recursive call results in this loop being applied to each vertex
- The total number of steps is the sum of the edges that originate at each vertex, which is the total number of edges, $|E|$
- The algorithm is $O(|E|)$

Minimum Spanning Tree

- The minimum spanning tree (MST) of a **weighted connected** graph is a subgraph that **contains all of the nodes** of the original graph and a subset of the edges so that:
 - The subgraph is connected
 - The total weights of the subgraph edges is the **smallest possible**

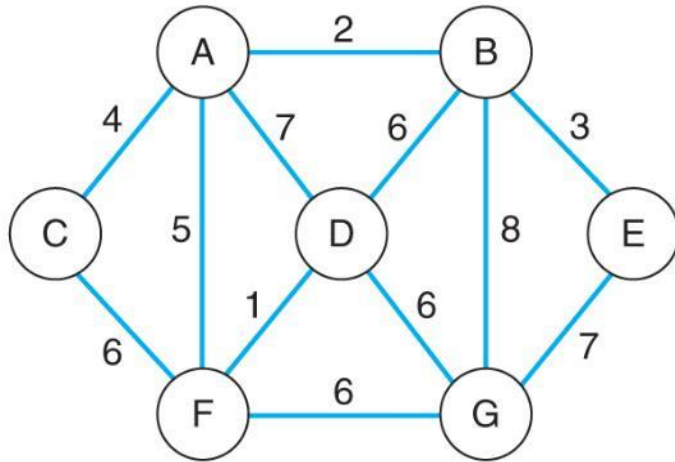
Brute Force Algorithm

- We could look at each subset of the edge set and first see if it is connected and then see if its total weight is smaller than the best answer so far
- If there are N edges in the graph, this process will require that we look at 2^N different subsets
- This is a lot more work than is needed

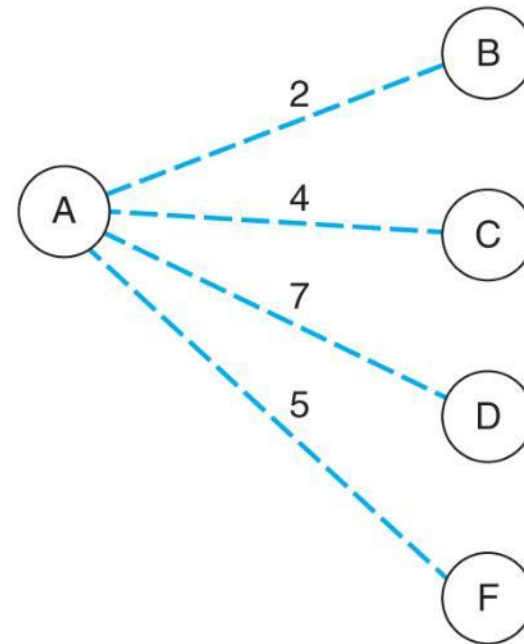
The Dijkstra-Prim Method

- This is a greedy algorithm that solves the larger problem by looking at a subset and making the best choice for it
- In this case, we will look at all of the edges from the starting node and choose the smallest
- On each pass, we will choose the smallest of the edges from nodes already in the **MST** to nodes not in the **MST** (the fringe)

Dijkstra-Prim Example

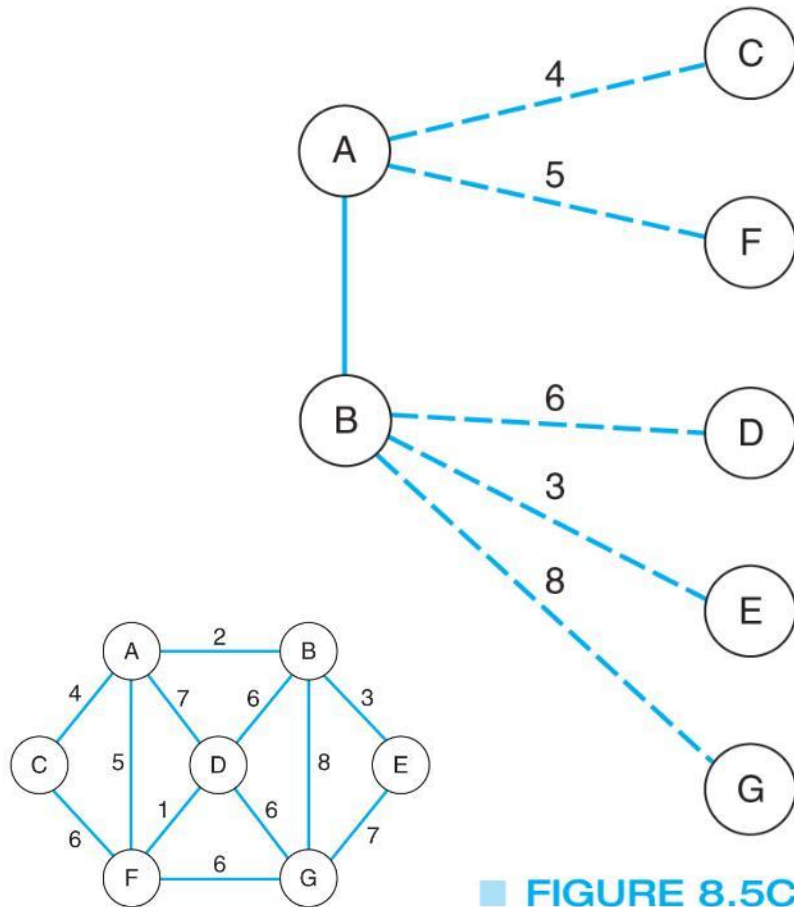


■ **FIGURE 8.5A**
The original graph

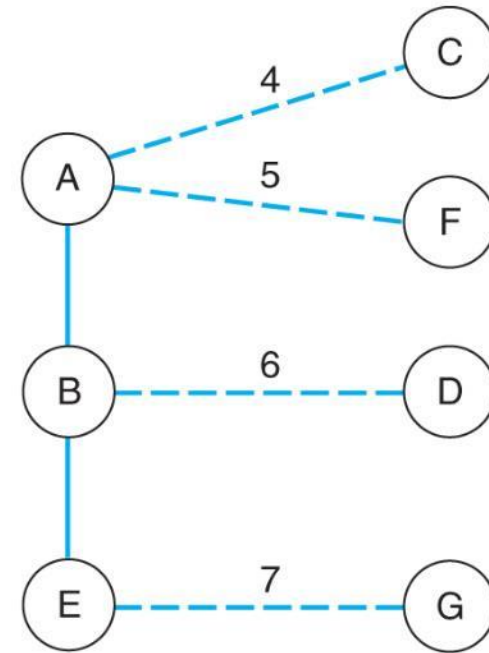


■ **FIGURE 8.5B**
First node added. (Dashed lines show edges to fringe nodes.)

Dijkstra-Prim Example

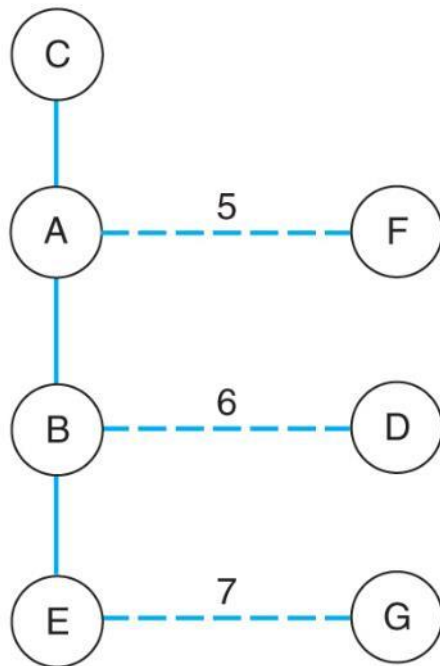


■ **FIGURE 8.5C** cond node added.
The original graph to nodes D, E, and
G updated. (Solid lines
show edges in the tree.)

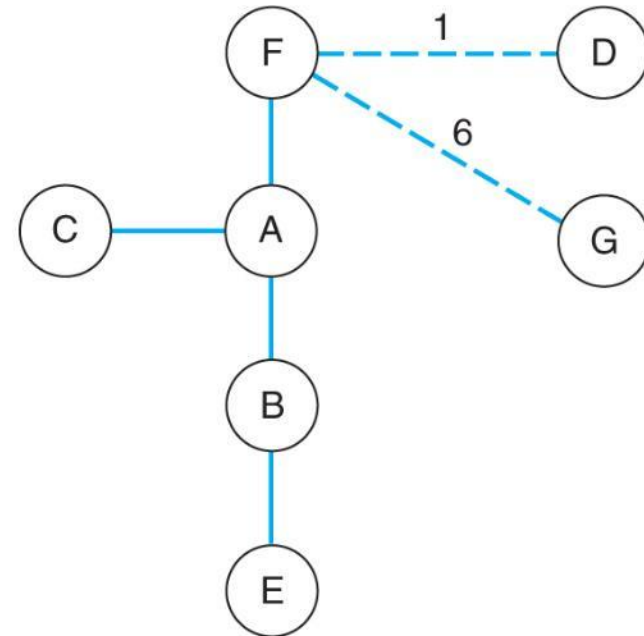


■ **FIGURE 8.5D**
Third node added. Edge
to node G updated.

Dijkstra-Prim Example

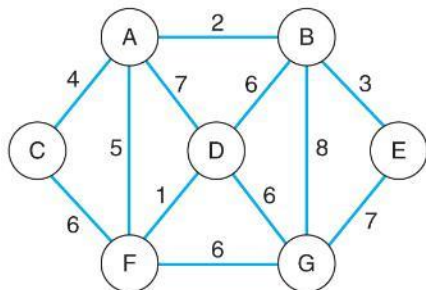


■ **FIGURE 8.5E**
Node C added to the tree



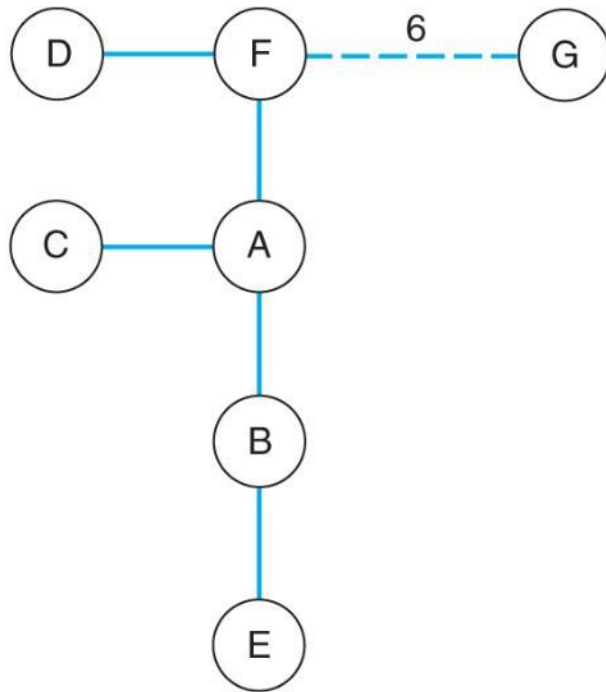
■ **FIGURE 8.5F**

Node F added to the tree. Edges to nodes D and G updated.

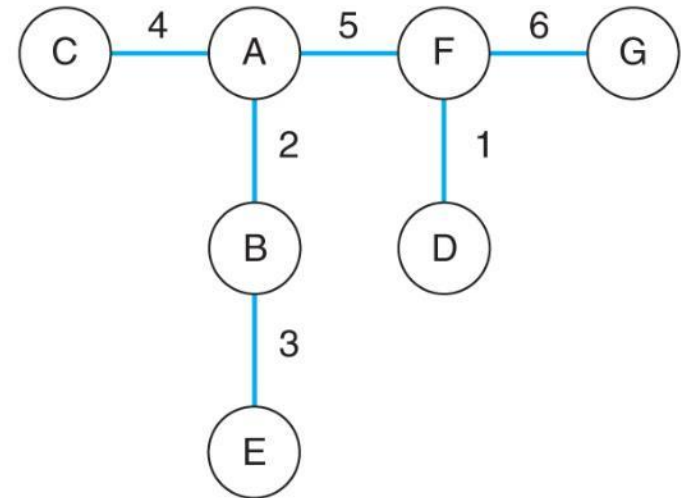


■ **FIGURE 8.6A**

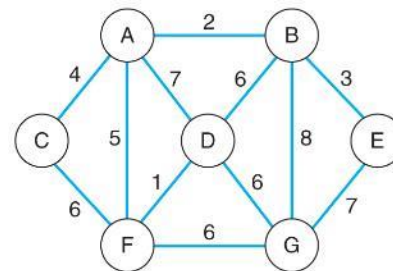
Dijkstra-Prim Example



■ **FIGURE 8.5G**
Only one node is left in the fringe



■ **FIGURE 8.5H**
The complete minimum spanning tree rooted at node A



■ **FIGURE 8.6A**
The original graph

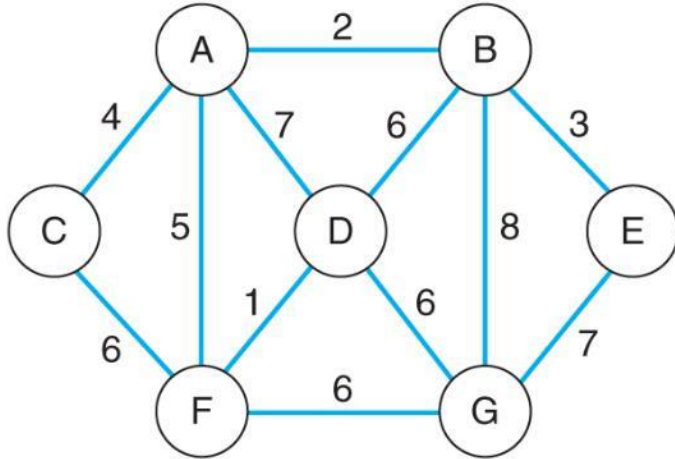
The Dijkstra-Prim Algorithm

```
Select a starting node
build the initial fringe from nodes
    connected to the starting node
while there are nodes left do
    choose the edge to the fringe of the smallest
        weight
    add the associated node to the tree
    update the fringe by:
        adding nodes to the fringe connected to the
            new node
        updating the edges to the fringe so that
            they are the smallest
end while
```

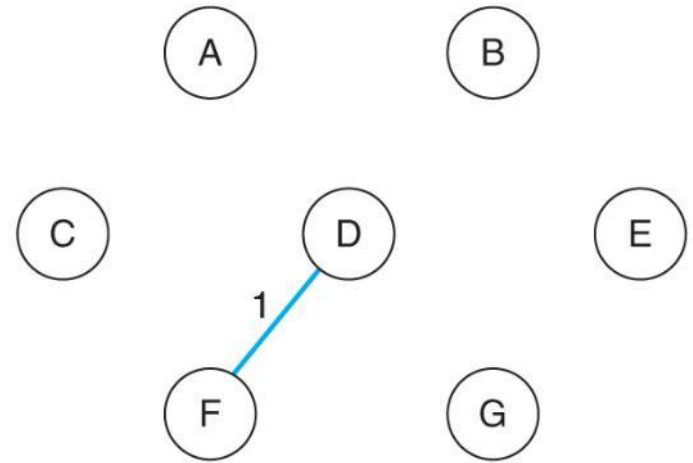
The Kruskal Method

- Kruskal's algorithm concentrates on the **edges** instead of the nodes
- First the edges are sorted in order of nondecreasing weight
- Edges are added to the MST as long as they do not form a **cycle with edges added in previous passes**
- When all of the nodes are connected and there are **$N - 1$** edges in the MST, we are done

Kruskal Example

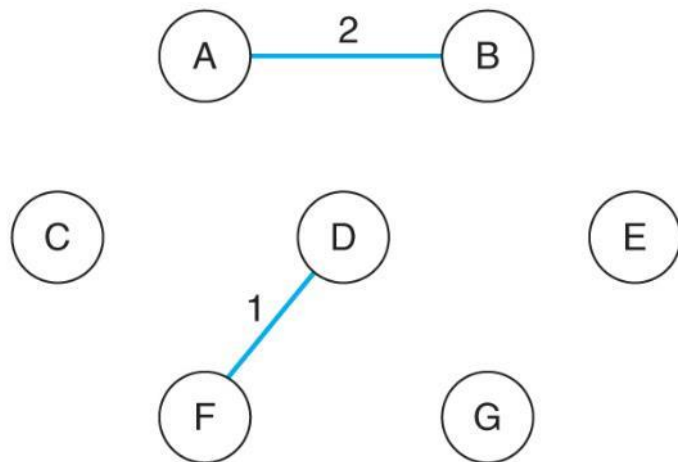


■ **FIGURE 8.6A**
The original graph

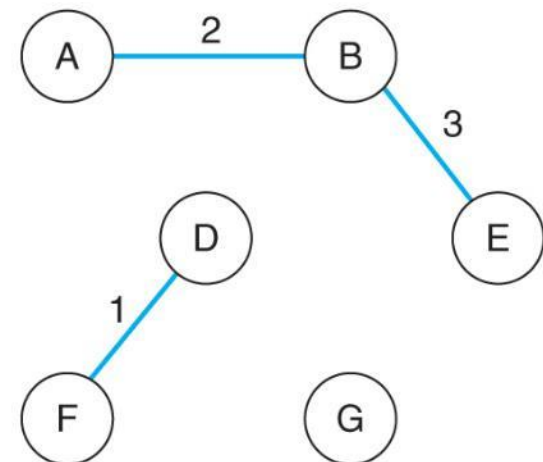


■ **FIGURE 8.6B**
First edge added

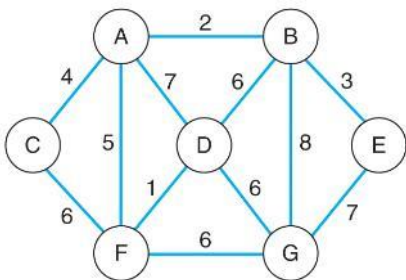
Kruskal Example



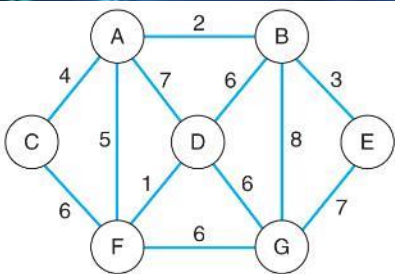
■ **FIGURE 8.6C**
Second edge added



■ **FIGURE 8.6D**
Third edge added

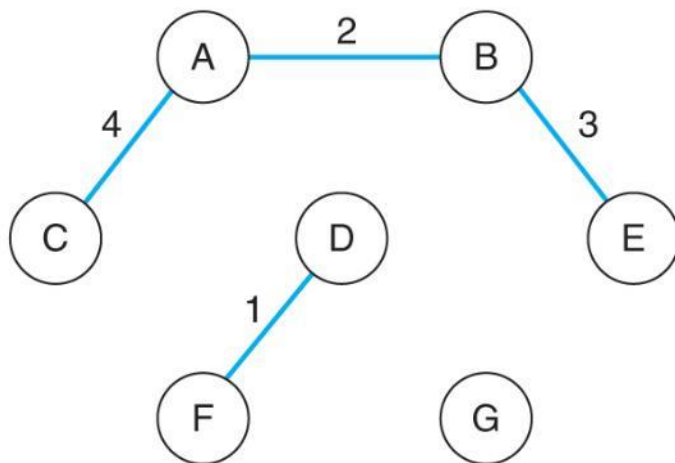


■ **FIGURE 8.6A**
The original graph

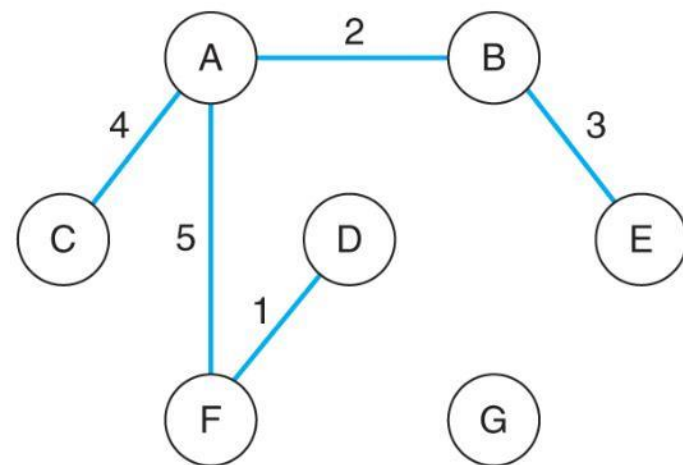


■ **FIGURE 8.6A**
The original graph

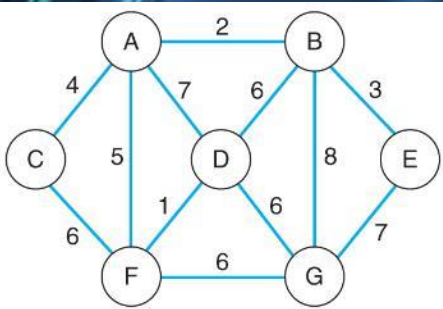
Kruskal Example



■ **FIGURE 8.6E**
Fourth edge added

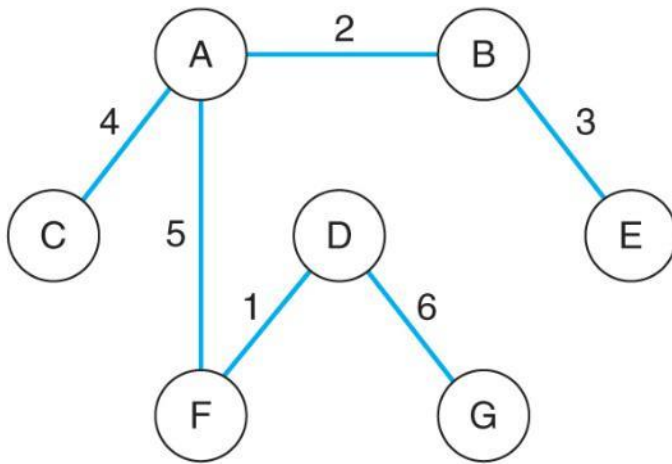


■ **FIGURE 8.6F**
Fifth edge added

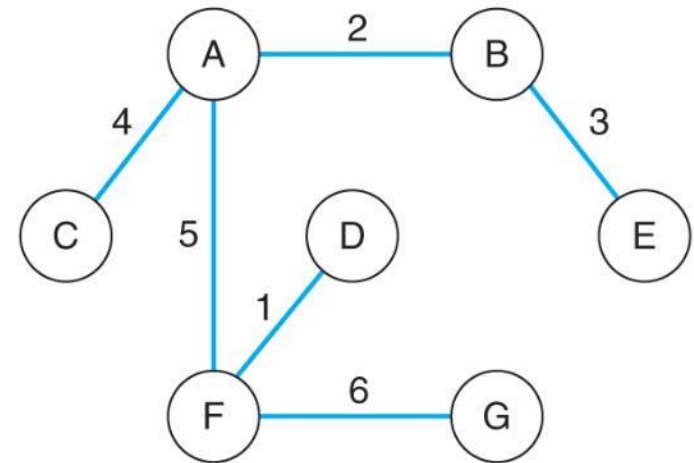


■ **FIGURE 8.6A**
The original graph

Kruskal Example



■ **FIGURE 8.6G**
A minimum spanning tree



■ **FIGURE 8.6H**
An alternative minimum spanning tree

The Kruskal Algorithm

```
sort the edges in nondecreasing order by weight
initialize partition structure
edgeCount = 1
includedCount = 0
while edgeCount  $\leq$  E and includedCount  $\leq$  N-1 do
    parent1 = FindRoot(edge[edgeCount].start )
    parent2 = FindRoot( edge[edgeCount].end )
    if parent1  $\neq$  parent2 then
        add edge[edgeCount] to spanning tree
        includedCount = includedCount + 1
        Union( parent1, parent2 )
    end if
    edgeCount = edgeCount + 1
end while
```



The Kruskal Algorithm

Complexity?

$O(E)$

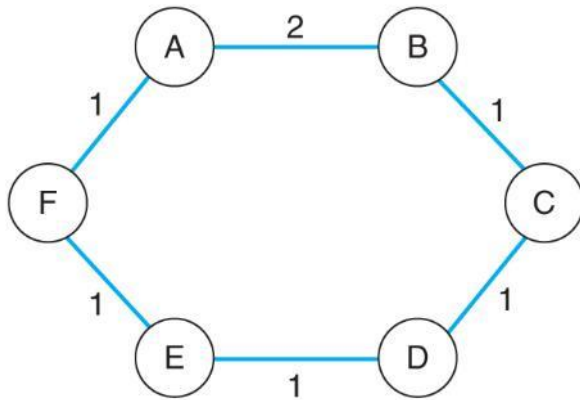


Shortest-Path Algorithm

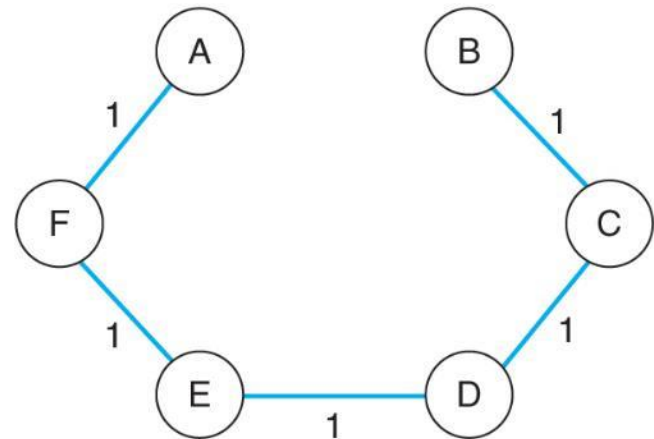
- The shortest-path algorithm finds the series of edges between two nodes that has the smallest total weight

MST vs. Shortest Path

- The MST algorithm can skip an edge of larger weight but include many edges of smaller weight that results in a longer path than the single edge



■ **FIGURE 8.7A**
A ring graph



■ **FIGURE 8.7B**
Its minimum spanning tree

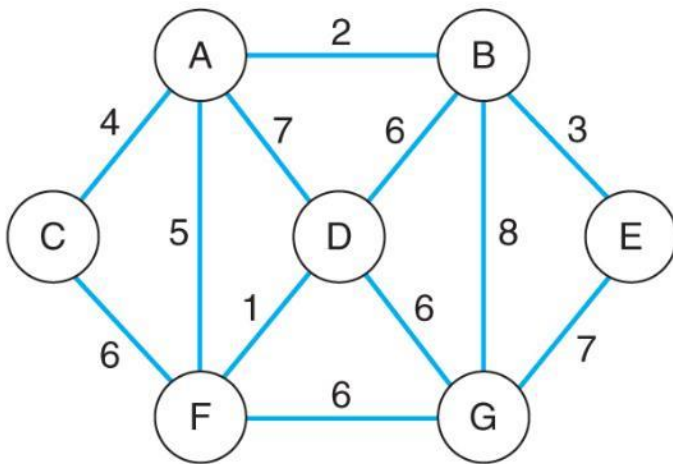
Dijkstra's Method

- This is like the Dijkstra-Prim MST algorithm, but:
 - Instead of just looking at the single shortest edge from a node to the fringe,
 - We look at the shortest path from the start node to a fringe node

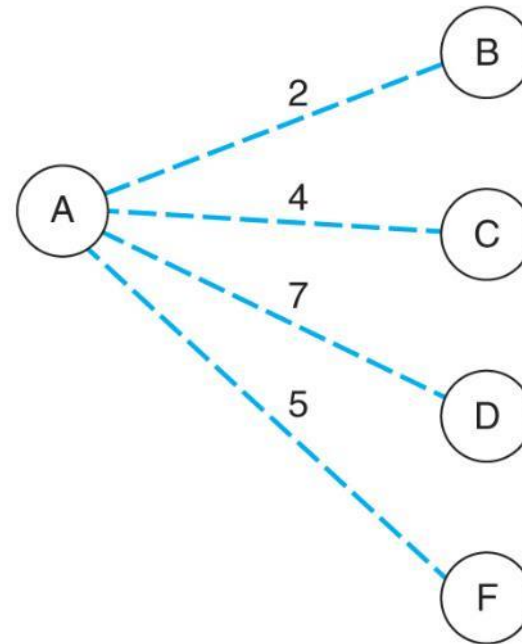
Dijkstra's Algorithm

```
Select a starting node
build the initial fringe from nodes
    connected to the starting node
while we are not at the destination node do
    choose the fringe node with the shortest path
        to the starting node
    add that node and its edge to the tree
    update the fringe by:
        adding nodes to the fringe connected to
            the new node
    for each node in the fringe do
        update its edge to the one connected to
            the tree on the shortest path to the
            starting node
    end for
end while
```

Dijkstra's Shortest-Path Example

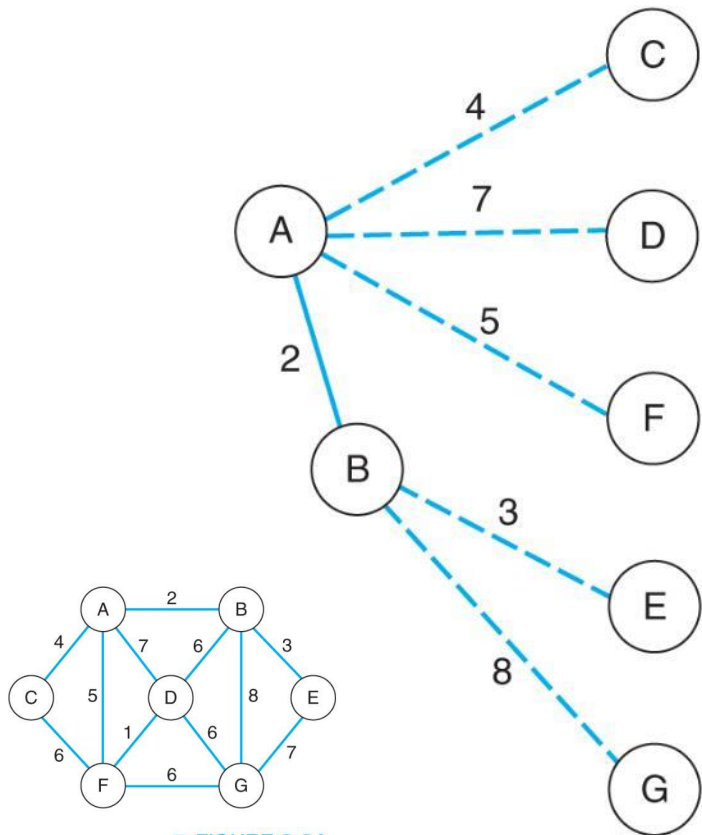


■ **FIGURE 8.8A**
The original graph



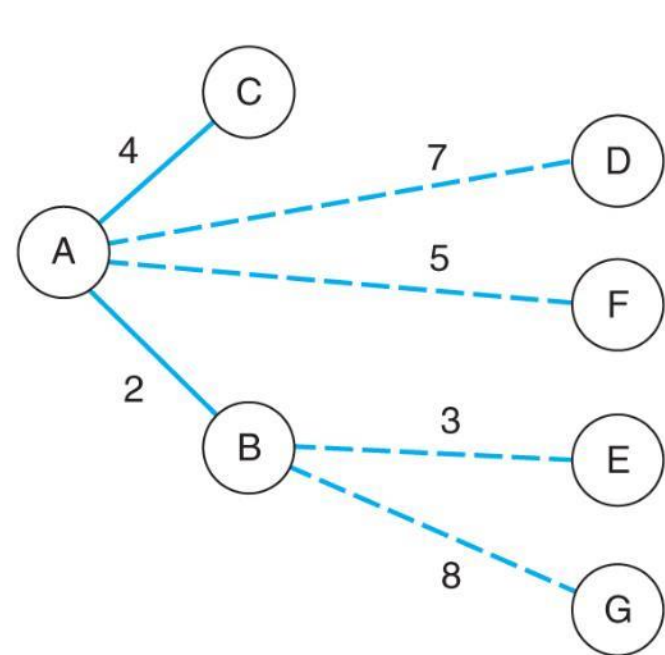
■ **FIGURE 8.8B**
The shortest path is to
node B

Dijkstra's Shortest-Path Example



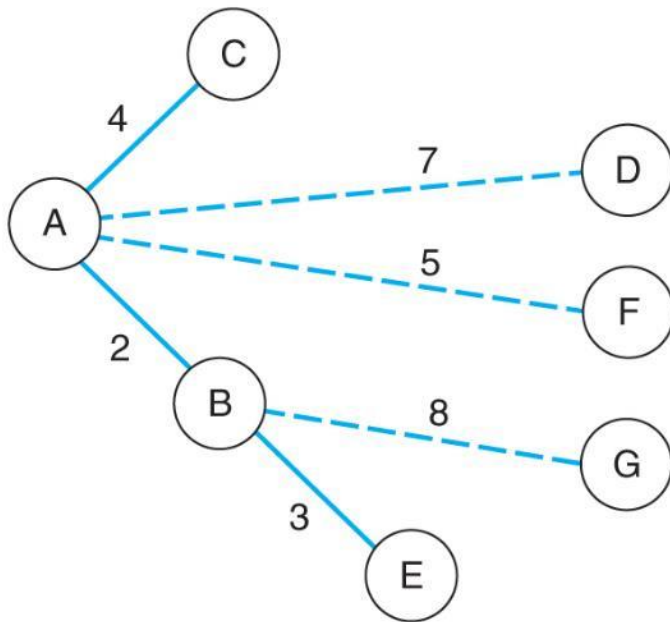
■ **FIGURE 8.8A**
The original graph

■ **FIGURE 8.8C**
Path of length 4 to node C is the shortest of the options



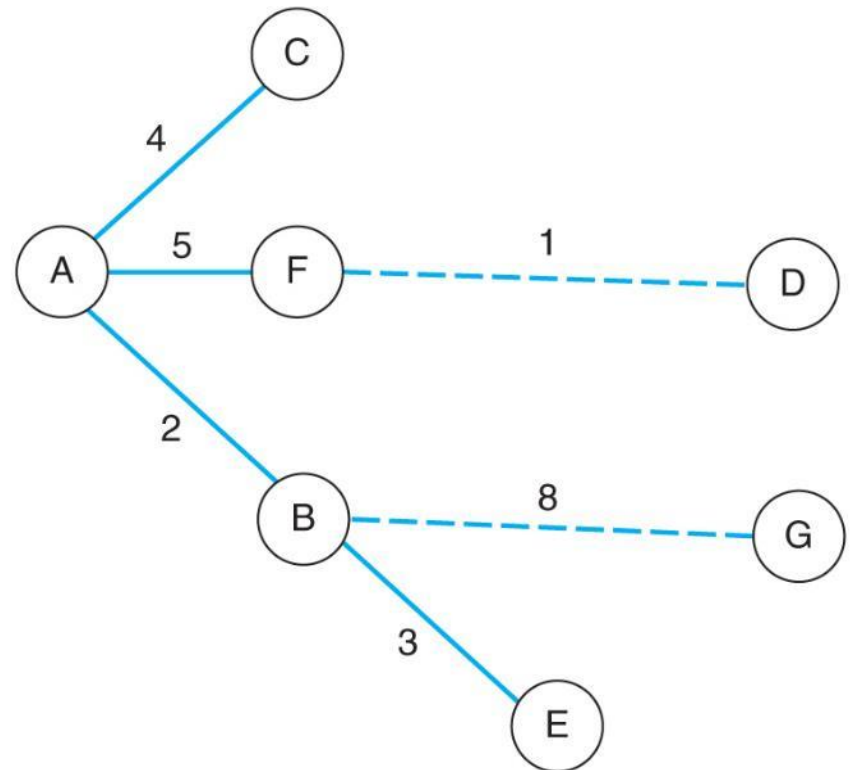
■ **FIGURE 8.8D**
The path of length 5 to either node E or node F is shortest

Dijkstra's Shortest-Path Example



■ **FIGURE 8.8E**

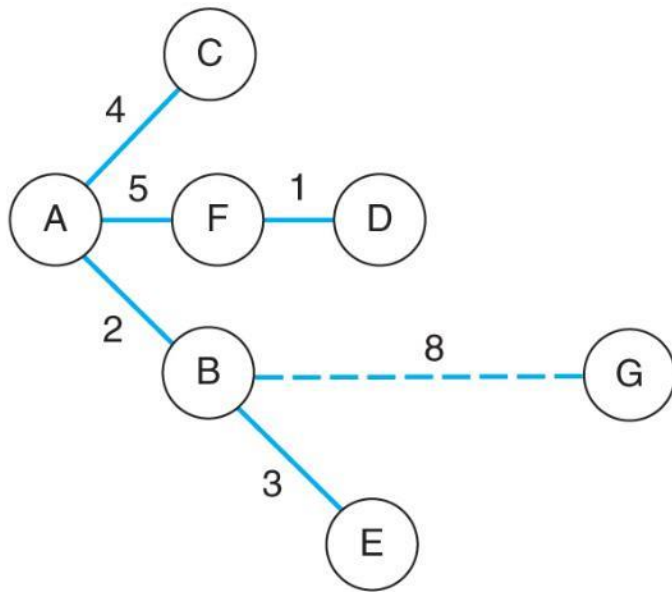
The other path of length 5 to node F is next



■ **FIGURE 8.8F**

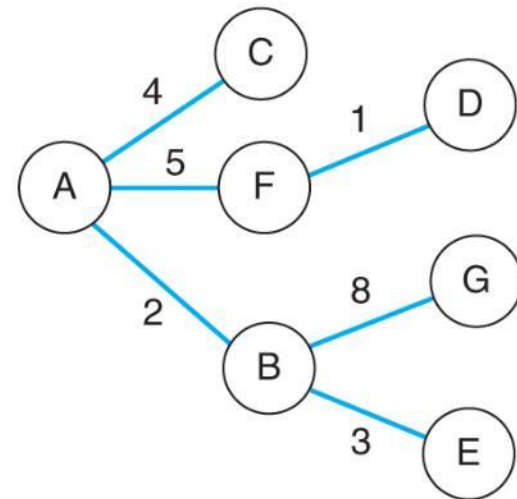
The path of length 6 to node D is shorter than the path to node G

Dijkstra's Shortest-Path Example



■ **FIGURE 8.8G**

The path to node G is the only one left



■ **FIGURE 8.8H**

The complete shortest path tree starting at node A

Finding the Shortest Path from a Vertex to All Other Vertices

- The breadth-first search found the shortest path from the start vertex to all other vertices, assuming that the length or weight of each edge was the same
- Dijkstra's algorithm finds the shortest path in a weighted directed graph

Finding the Shortest Path from a Vertex to All Other Vertices (cont.)

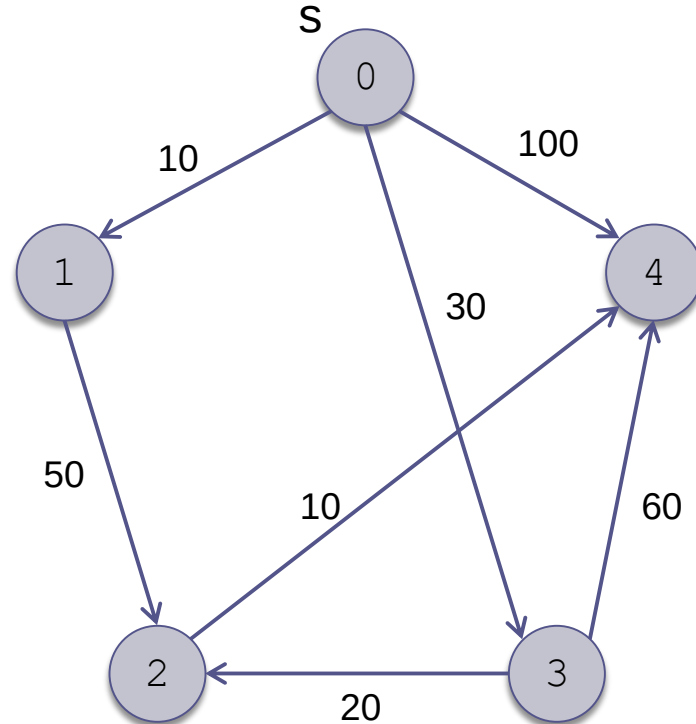
- We need 2 sets and 2 arrays
 - ▣ Set S will contain the vertices for which we have computed the shortest distance
 - Initialize S by placing the starting vertex s into it
 - ▣ Set $V-S$ will contain the vertices we still need to process
 - Initialize $V-S$ by placing the remaining vertices into it
 - ▣ $d[v]$ will contain the shortest distance from s to v
 - For each V in $V-S$, set $d[v]$ to the weight of the edge $w(s,v)$ for each vertex v adjacent to s and to ∞ for the other vertices
 - ▣ $p[v]$ will contain the predecessor of v in the path from s to v
 - Initialize $p[v]$ to s for each v in $V-S$

Dijkstra's Algorithm

$S = \{\}$

$V-S = \{\}$

v	d[v]	p[v]
1		
2		
3		
4		

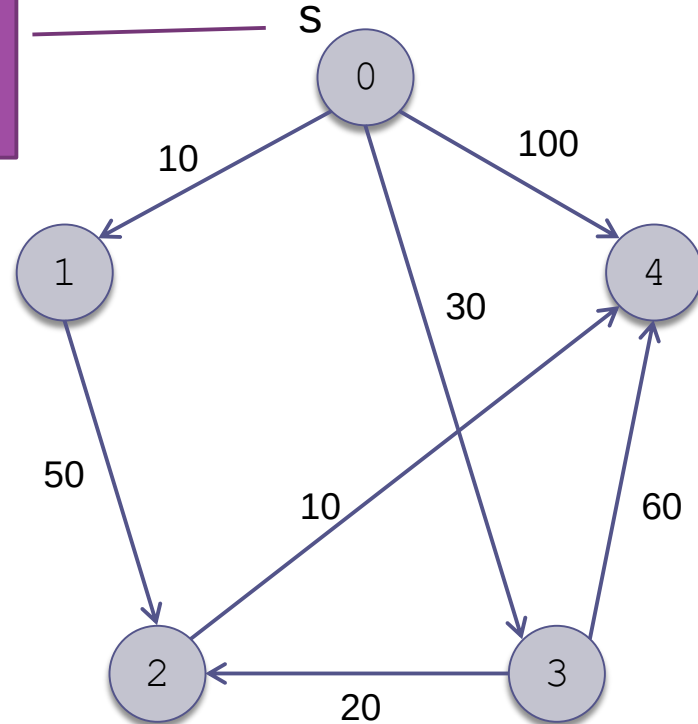


Dijkstra's Algorithm (cont.)

$S = \{\}$

$V-S = \{\}$

s is the start
vertex



v	d[v]	p[v]
1		
2		
3		
4		

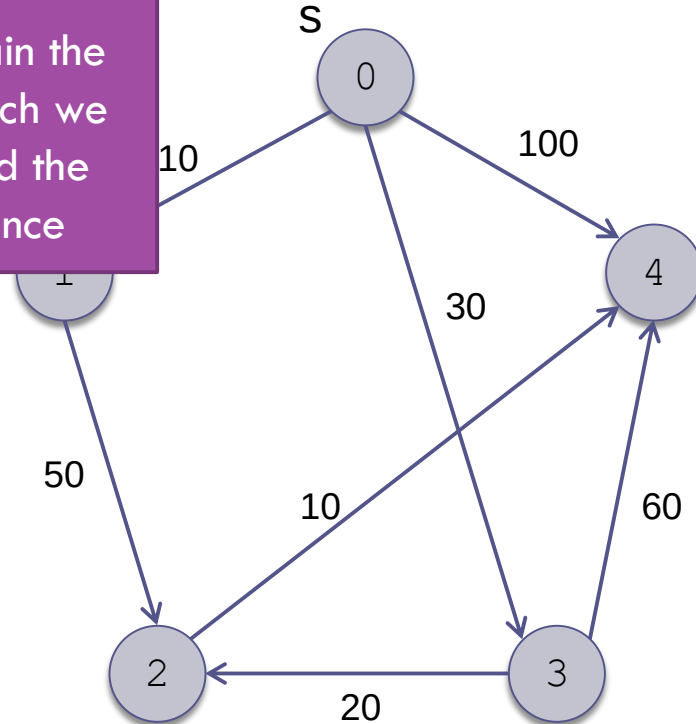
Dijkstra's Algorithm (cont.)

$S = \{\}$

$V - S = \{\}$

Set S will contain the vertices for which we have computed the shortest distance

v	$d[v]$	$p[v]$
1		
2		
3		
4		



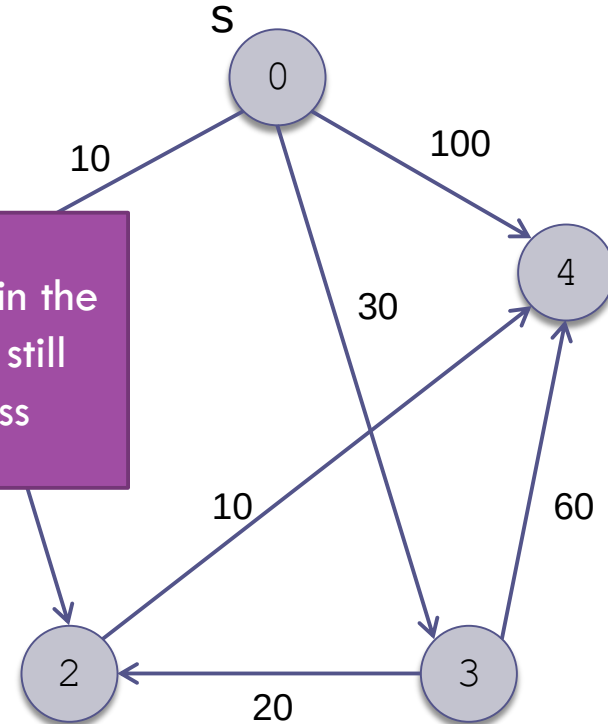
Dijkstra's Algorithm (cont.)

$S = \{\}$

$V-S = \{\}$

v	d[v]	p[v]
1		
2		
3		
4		

Set V-S will contain the vertices that we still need to process

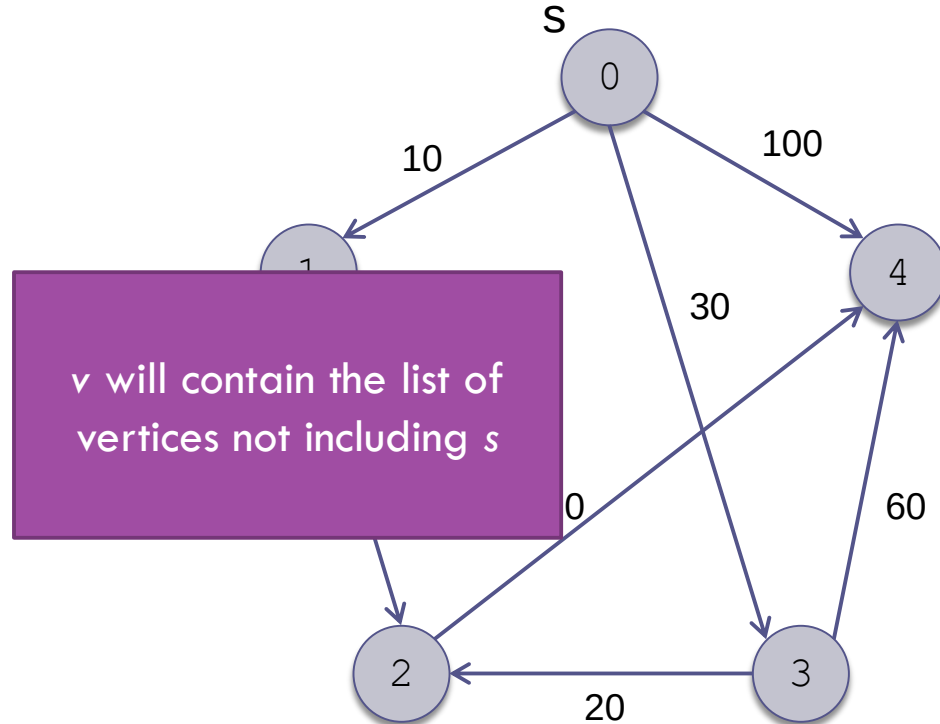


Dijkstra's Algorithm (cont.)

$S = \{\}$

$V-S = \{\}$

v	d[v]	p[v]
1		
2		
3		
4		

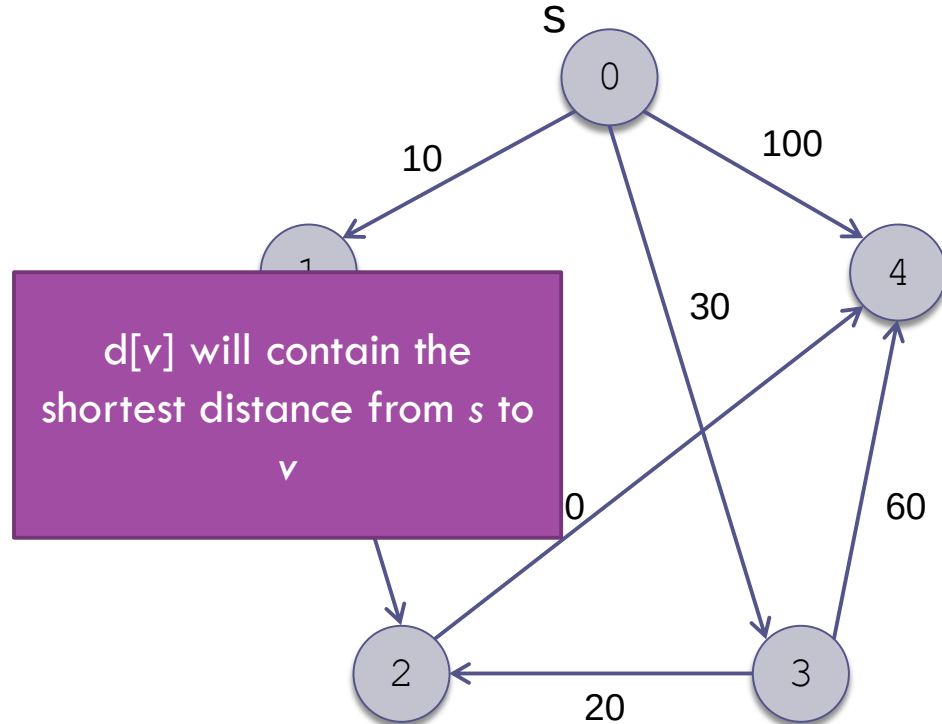


Dijkstra's Algorithm (cont.)

$S = \{\}$

$V-S = \{\}$

v	d[v]	p[v]
1		
2		
3		
4		

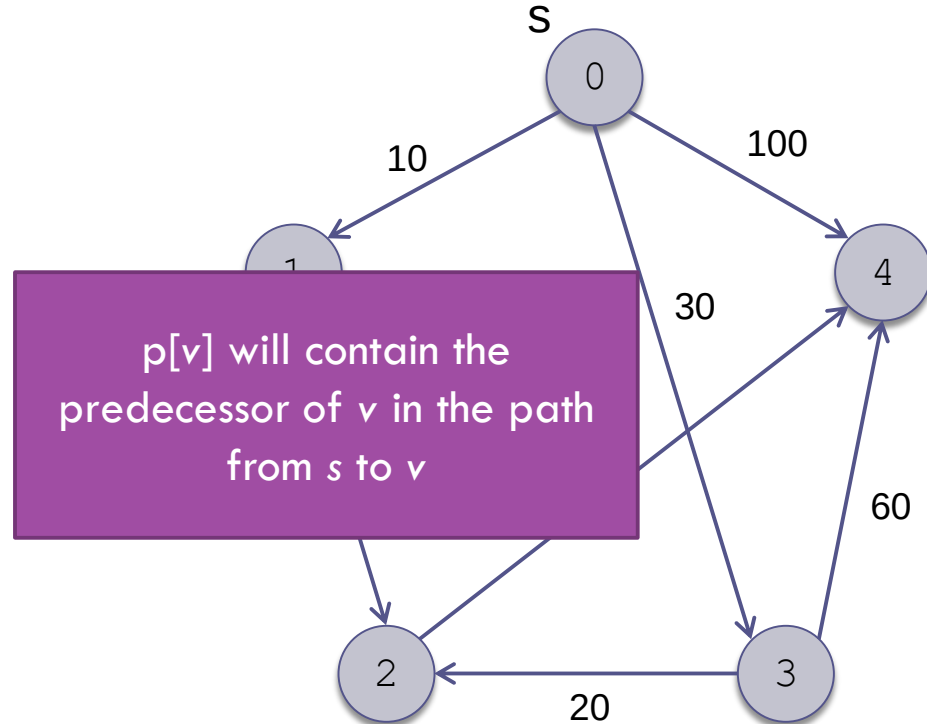


Dijkstra's Algorithm (cont.)

$S = \{\}$

$V-S = \{\}$

v	d[v]	p[v]
1		
2		
3		
4		

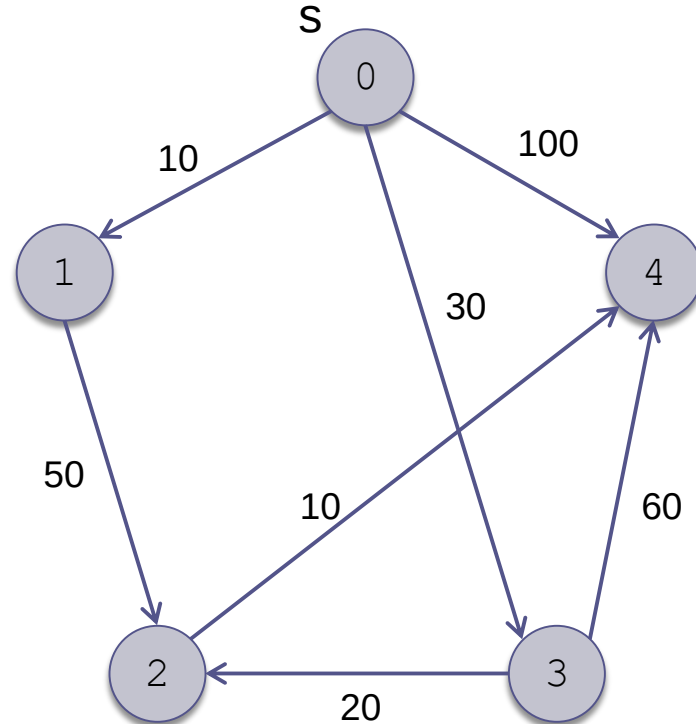


Dijkstra's Algorithm (cont.)

$S = \{ 0 \}$

$V-S = \{ 1, 2, 3, 4 \}$

v	d[v]	p[v]
1		
2		
3		
4		



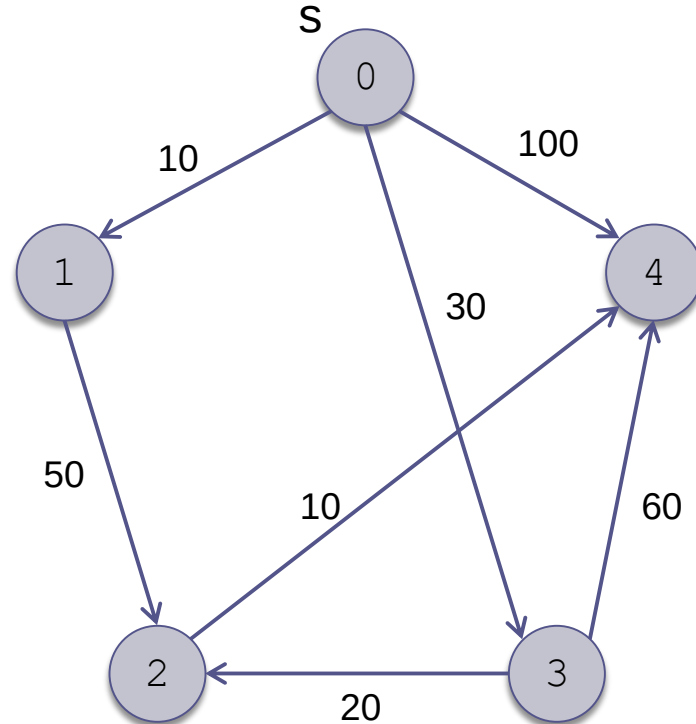
At initialization, the start vertex s is placed in S , and the remaining vertices into $V-S$

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1		
2		
3		
4		



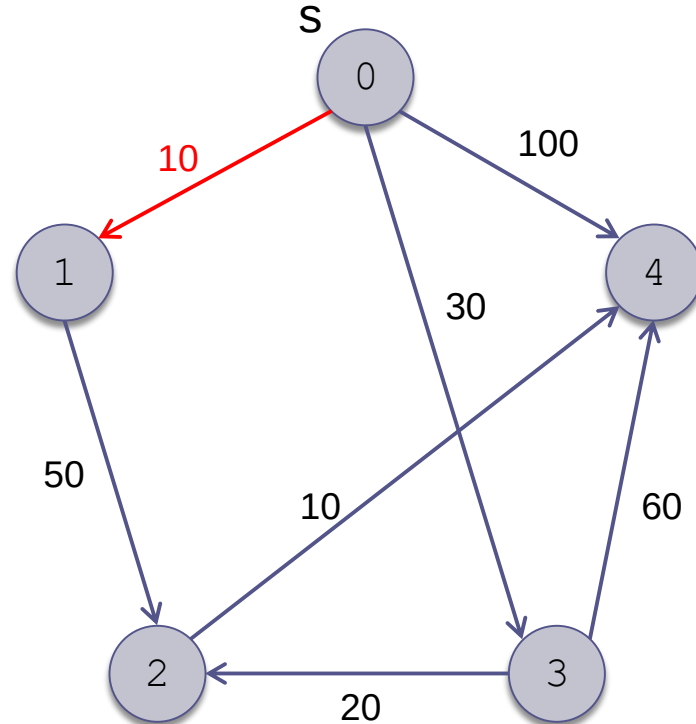
For each v in $V-S$, we initialize d by setting $d[v]$ equal to the weight of the edge $w(s, v)$ for each vertex v , adjacent to s

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1	10	
2		
3		
4		



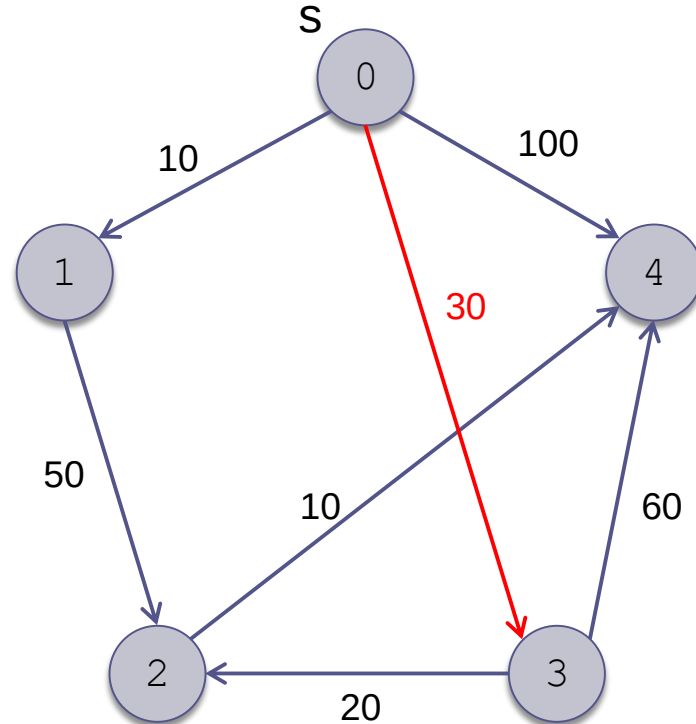
For each v in $V-S$, we initialize d by setting $d[v]$ equal to the weight of the edge $w(s, v)$ for each vertex v , adjacent to s

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1	10	
2		
3	30	
4		



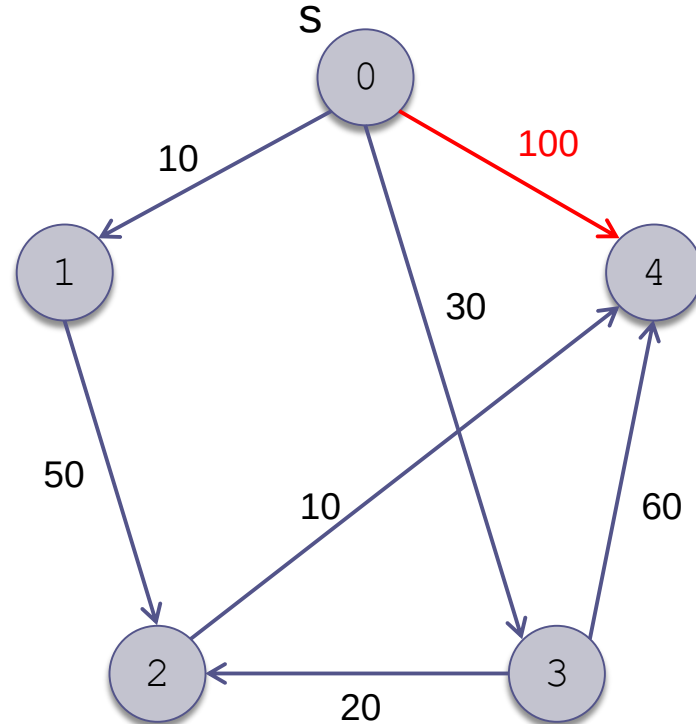
For each v in $V-S$, we initialize d by setting $d[v]$ equal to the weight of the edge $w(s, v)$ for each vertex v , adjacent to s

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1	10	
2		
3	30	
4	100	



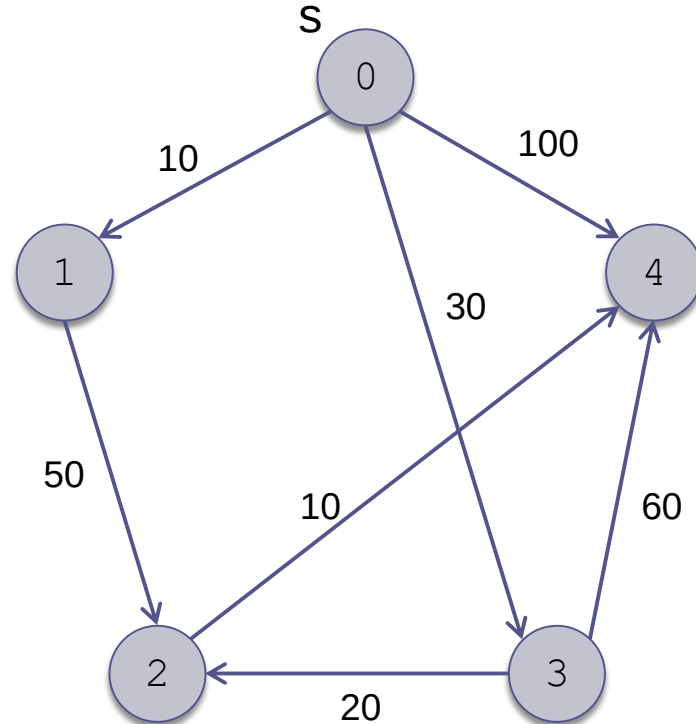
For each v in $V-S$, we initialize d by setting $d[v]$ equal to the weight of the edge $w(s, v)$ for each vertex v , adjacent to s

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V - S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1	10	
2		
3	30	
4	100	



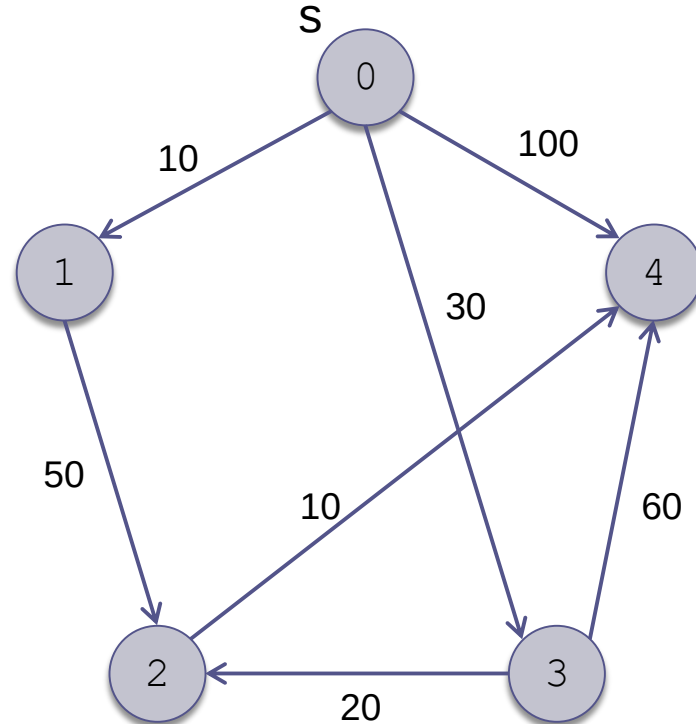
For each v not adjacent to s , we set $d[v]$ equal to ∞

Dijkstra's Algorithm (cont.)

$S = \{ 0 \}$

$V-S = \{ 1, 2, 3, 4 \}$

v	d[v]	p[v]
1	10	
2	∞	
3	30	
4	100	



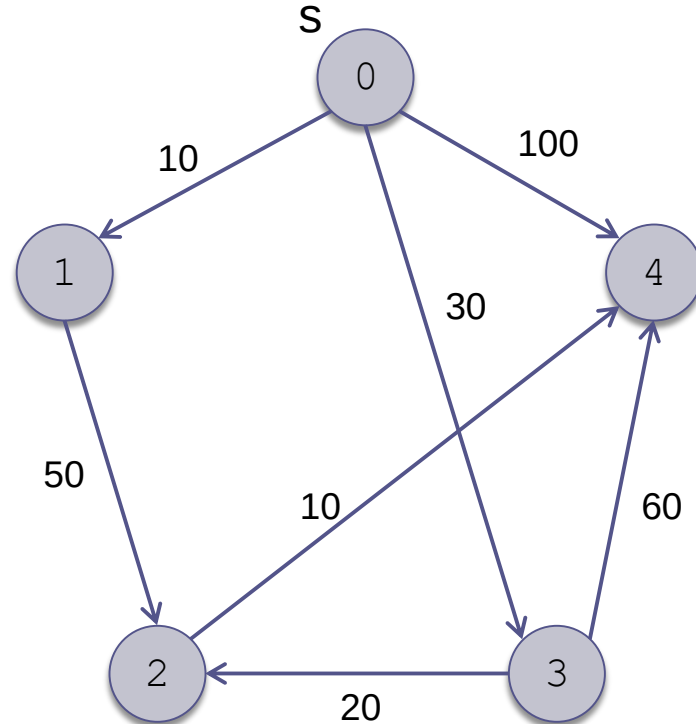
For each v not adjacent to s , we set $d[v]$ equal to ∞

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1	10	
2	∞	
3	30	
4	100	



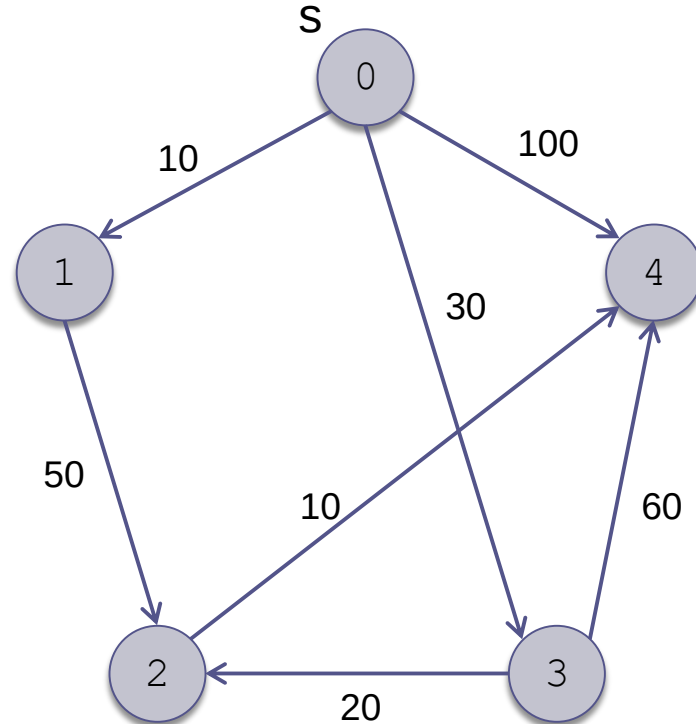
We initialize each $p[v]$ to s

Dijkstra's Algorithm (cont.)

$S = \{ 0 \}$

$V-S = \{ 1, 2, 3, 4 \}$

v	d[v]	p[v]
1	10	0
2	∞	0
3	30	0
4	100	0



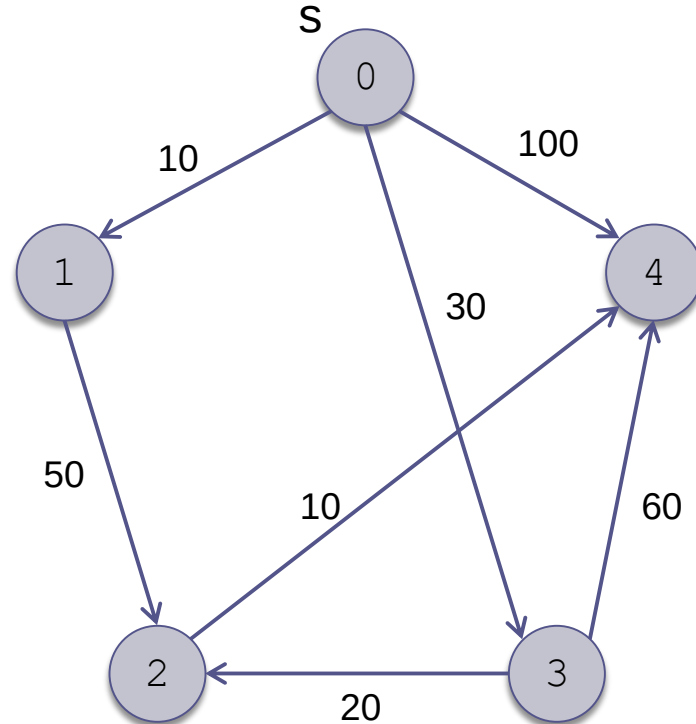
We initialize each $p[v]$ to s

Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

v	d[v]	p[v]
1	10	0
2	∞	0
3	30	0
4	100	0



We now find the vertex u in $V-S$ that has the smallest value of $d[u]$

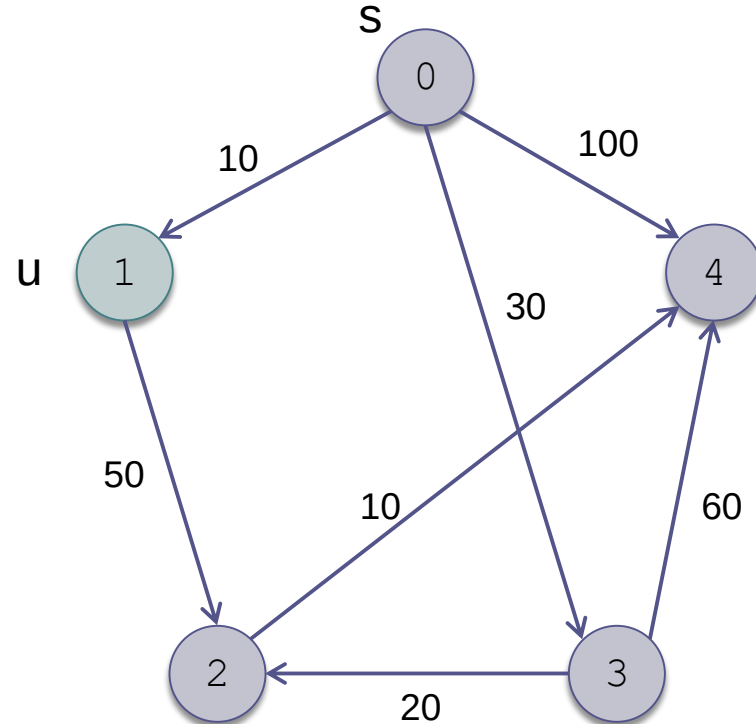
Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

$u = 1$

v	d[v]	p[v]
1	10	0
2	∞	0
3	30	0
4	100	0



We now find the vertex u in $V-S$ that has the smallest value of $d[u]$

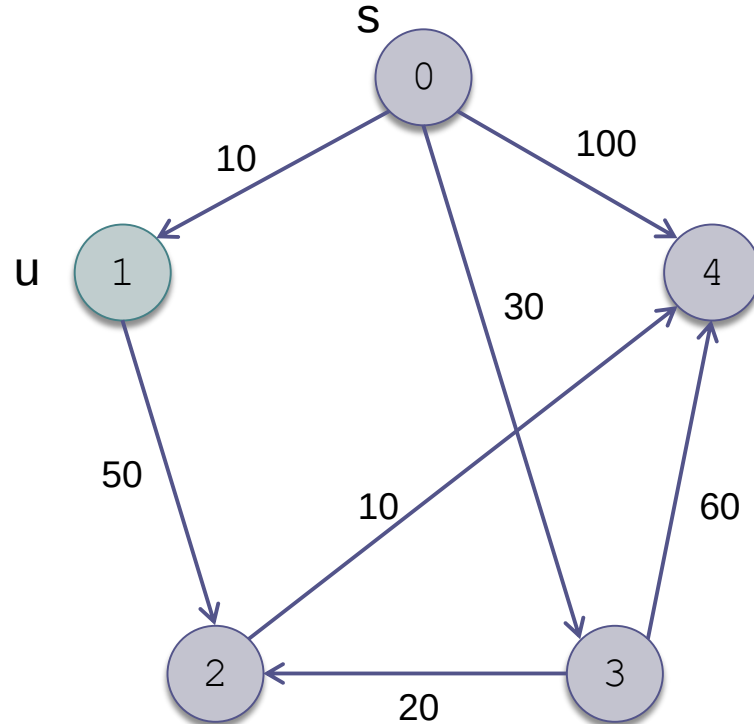
Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V - S = \{1, 2, 3, 4\}$

$u = 1$

v	$d[v]$	$p[v]$
1	10	0
2	∞	0
3	30	0
4	100	0



Consider the vertices v that are adjacent to u

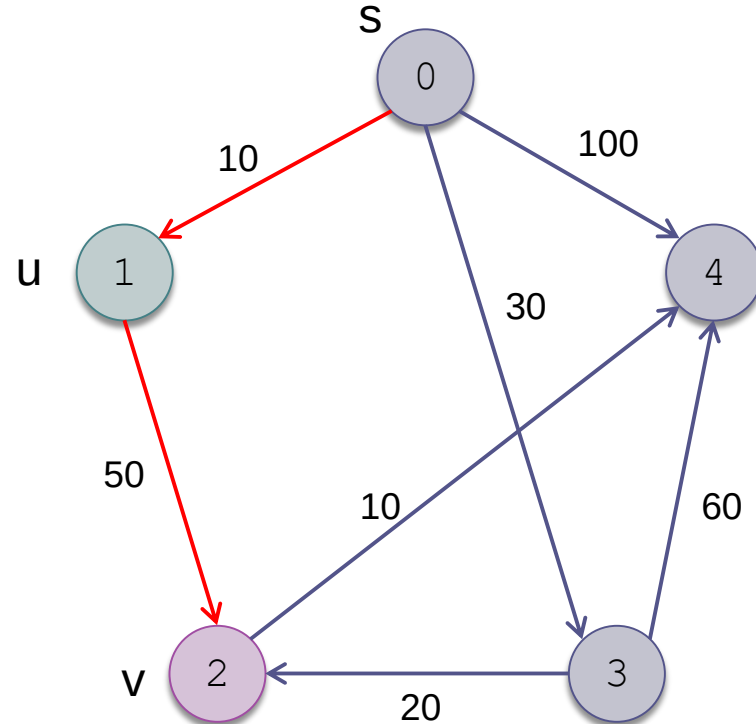
Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V - S = \{1, 2, 3, 4\}$

$u = 1$

v	$d[v]$	$p[v]$
1	10	0
2	∞	0
3	30	0
4	100	0



If the distance from s to u ($d[u]$) plus the distance from u to v is smaller than $d[v]$ we update $d[v]$ to that value

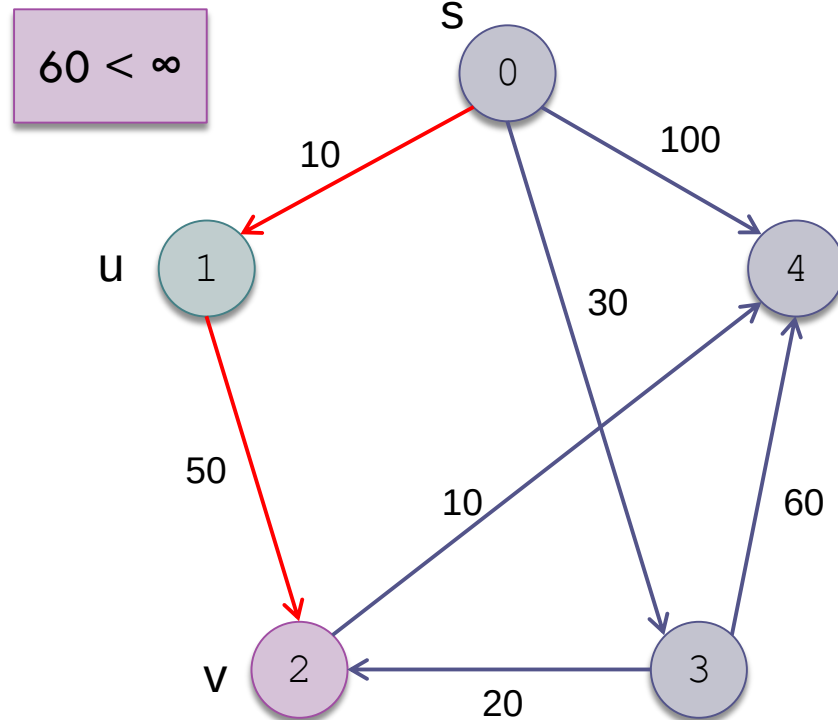
Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V - S = \{1, 2, 3, 4\}$

$u = 1$

v	$d[v]$	$p[v]$
1	10	0
2	∞	0
3	30	0
4	100	0



If the distance from s to u ($d[u]$) plus the distance from u to v is smaller than $d[v]$ we update $d[v]$ to that value

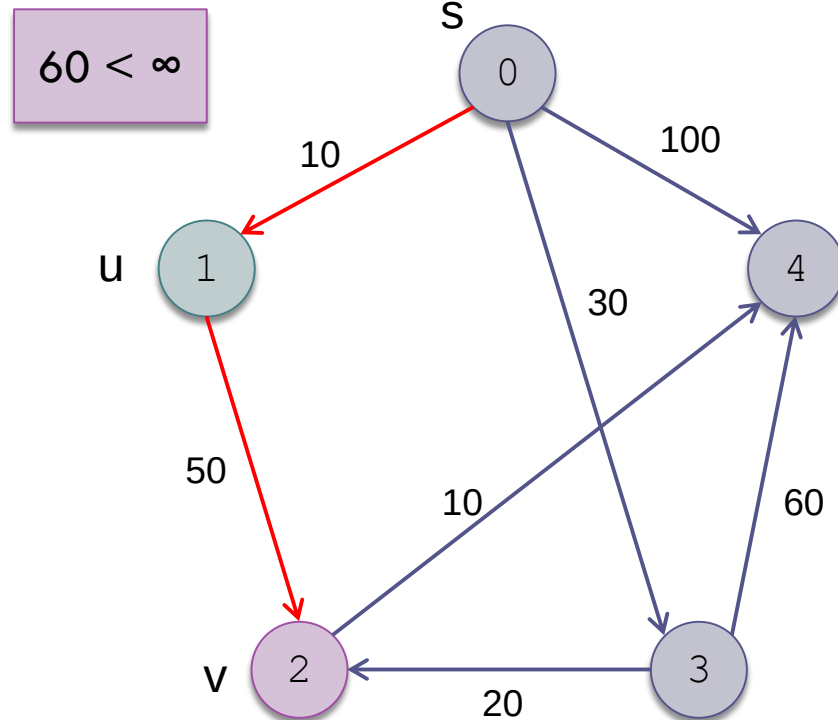
Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V - S = \{1, 2, 3, 4\}$

$u = 1$

v	$d[v]$	$p[v]$
1	10	0
2	60	0
3	30	0
4	100	0



If the distance from s to u ($d[u]$) plus the distance from u to v is smaller than $d[v]$ we update $d[v]$ to that value

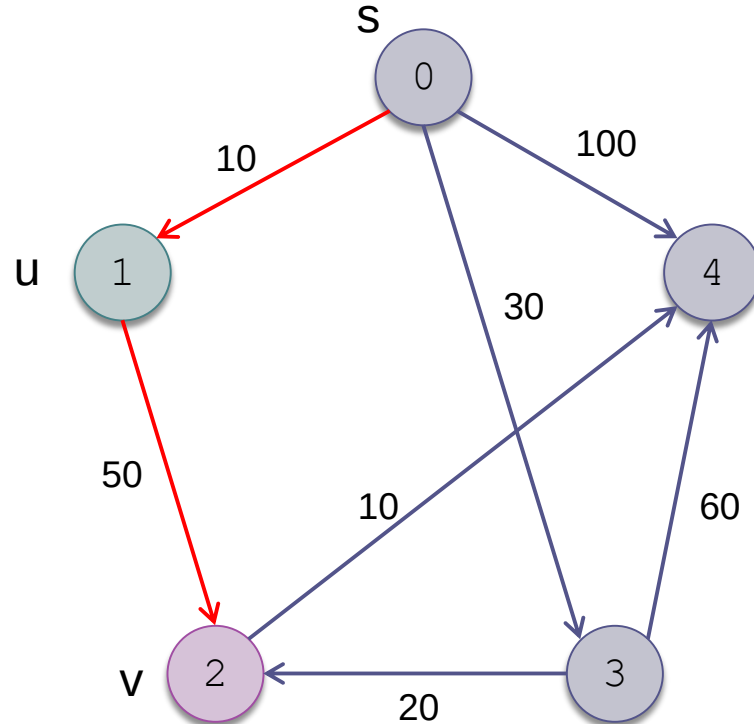
Dijkstra's Algorithm (cont.)

$S = \{ 0 \}$

$V - S = \{ 1, 2, 3, 4 \}$

$u = 1$

v	$d[v]$	$p[v]$
1	10	0
2	60	1
3	30	0
4	100	0



and set $p[v]$ to u

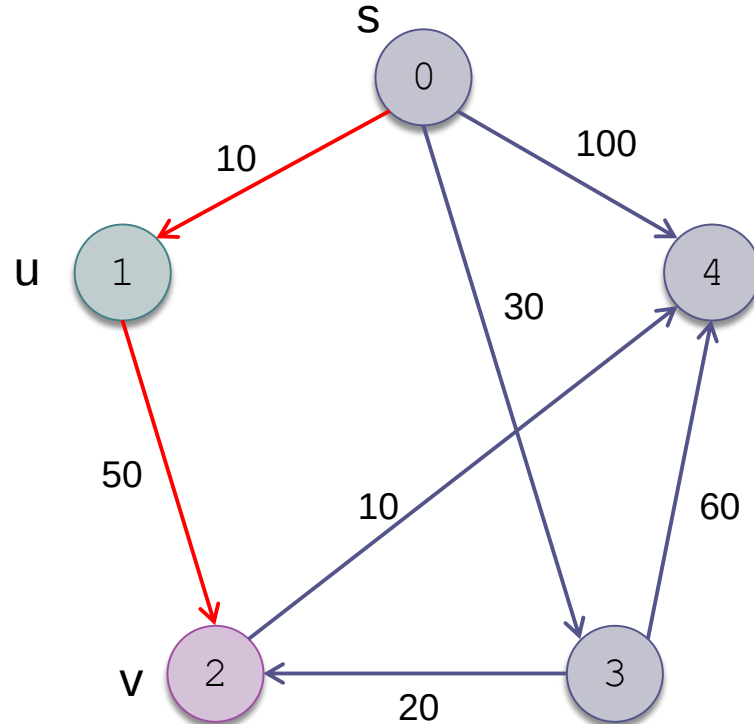
Dijkstra's Algorithm (cont.)

$S = \{0\}$

$V-S = \{1, 2, 3, 4\}$

$u = 1$

v	$d[v]$	$p[v]$
1	10	0
2	60	1
3	30	0
4	100	0



Remove u from $V-S$ and
place it in S

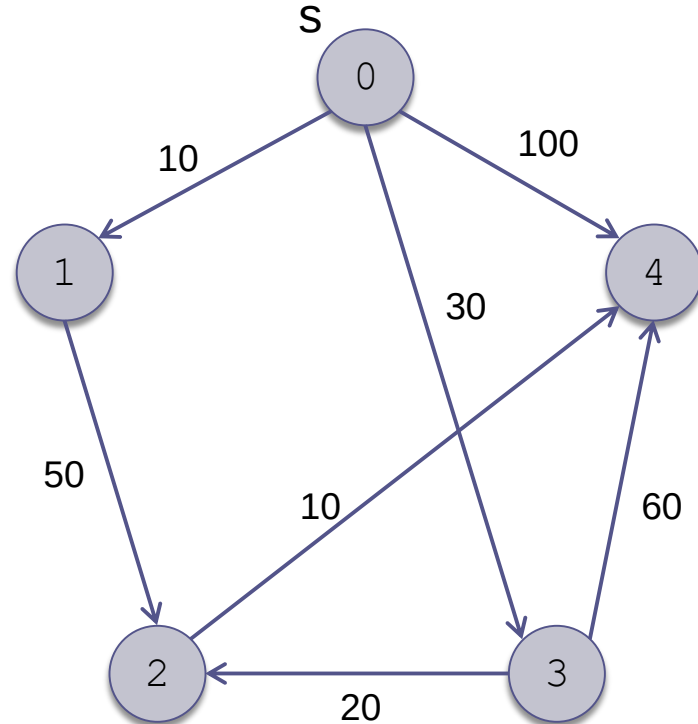
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 1$

v	d[v]	p[v]
1	10	0
2	60	1
3	30	0
4	100	0



Repeat until $V-S$ is empty

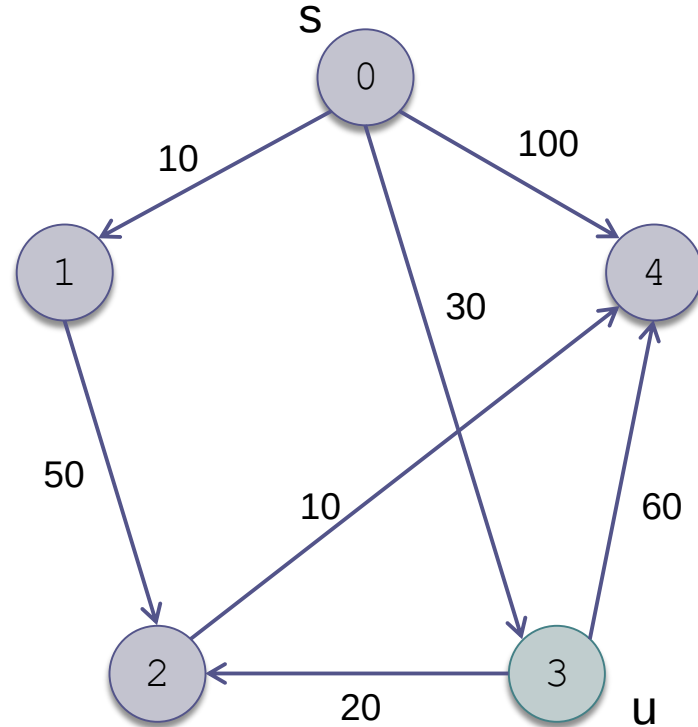
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	60	1
3	30	0
4	100	0



The smallest $d[v]$ in $V-S$ is
vertex 3

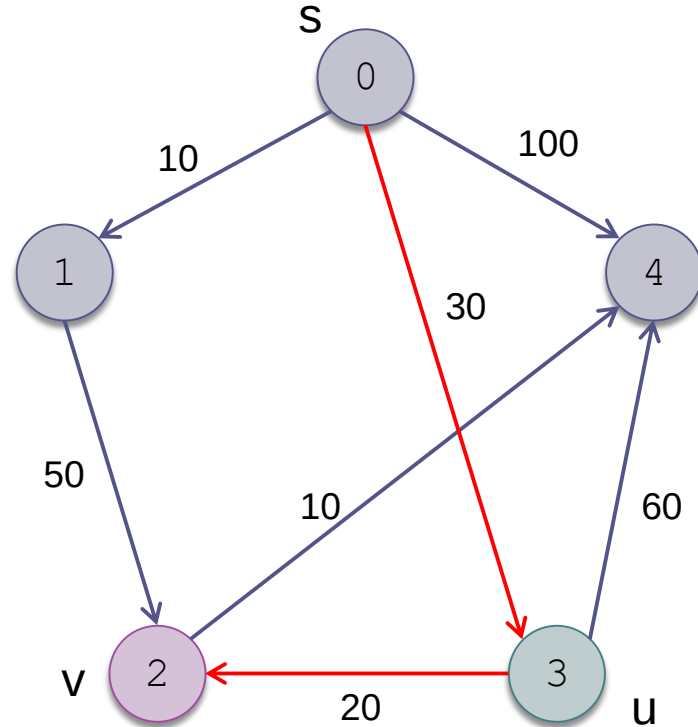
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	60	1
3	30	0
4	100	0



The distance from s to u plus the distance from u to v is 50

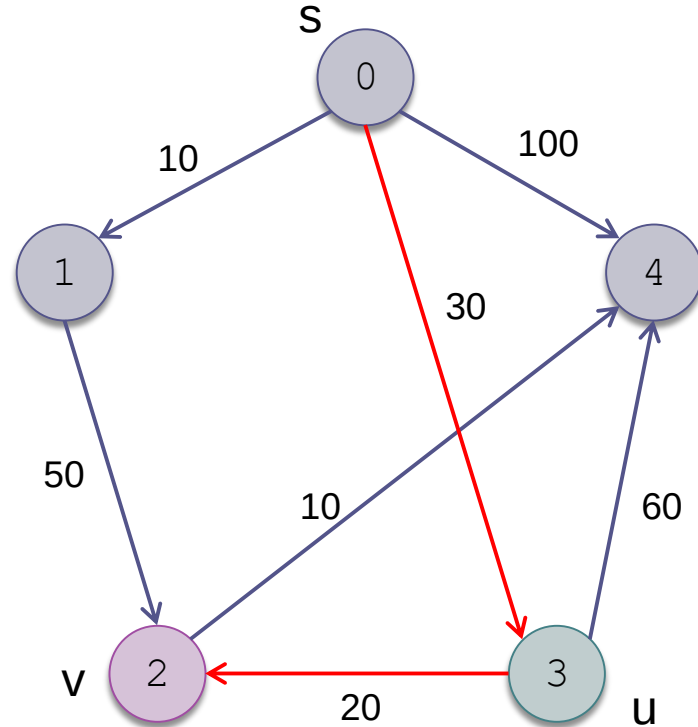
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	60	1
3	30	0
4	100	0



$50 < d[2]$ (which is 60)

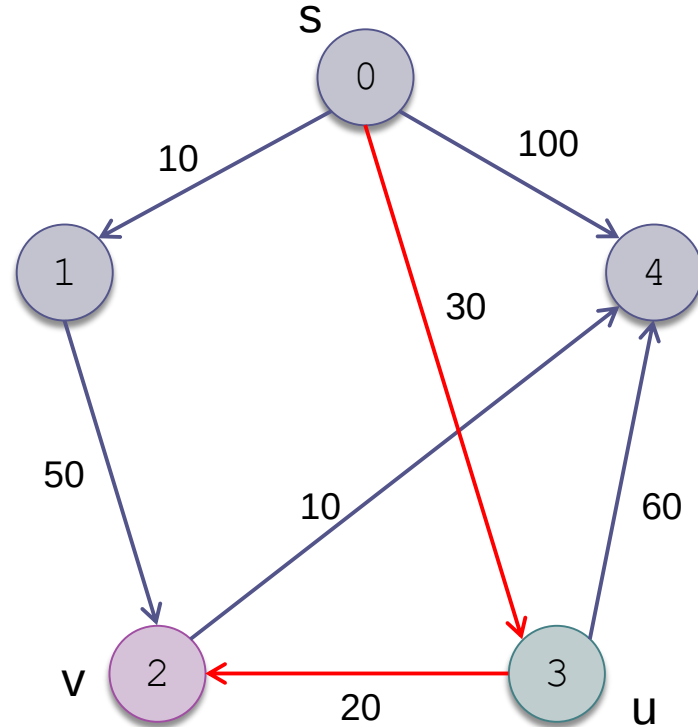
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	100	0



Set $d[2]$ to 50 and $p[2]$ to u

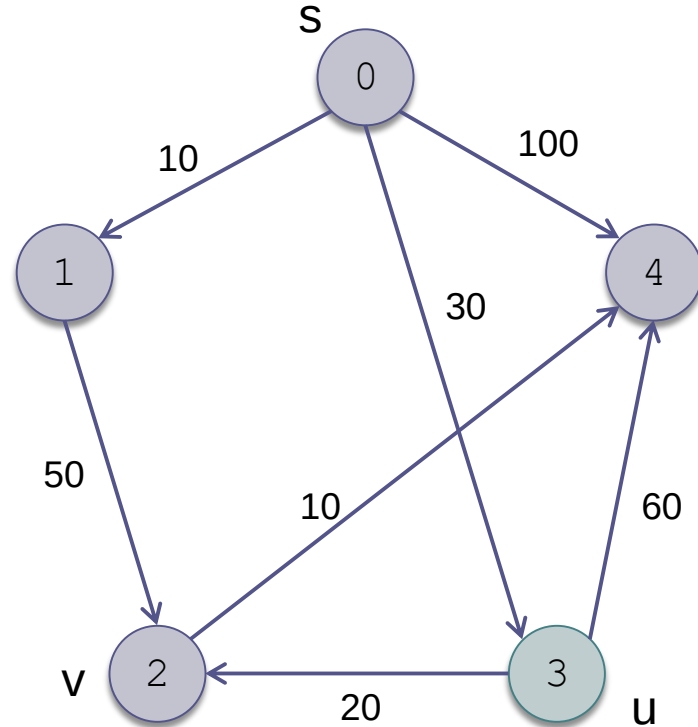
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	100	0



Continue to the next
adjacent vertex

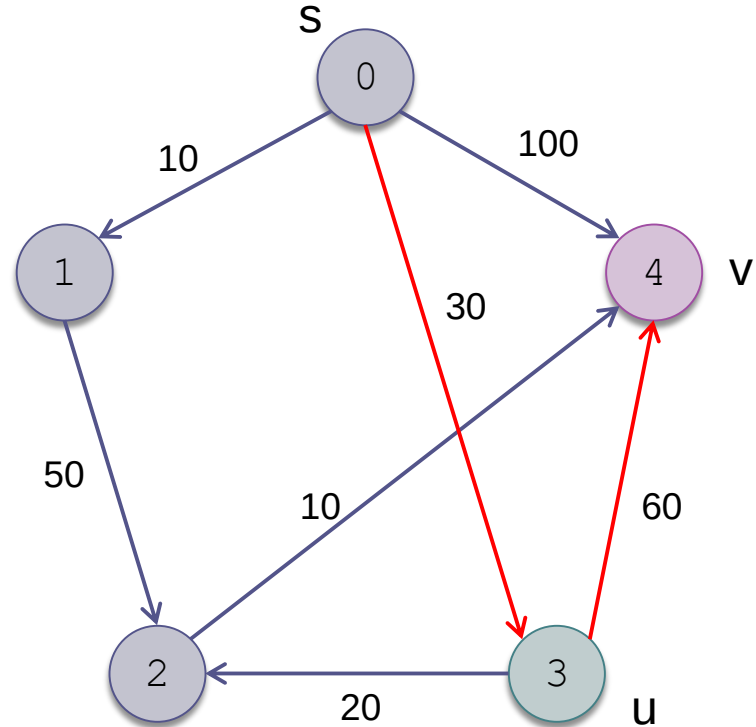
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V - S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	100	0



Continue to the next
adjacent vertex

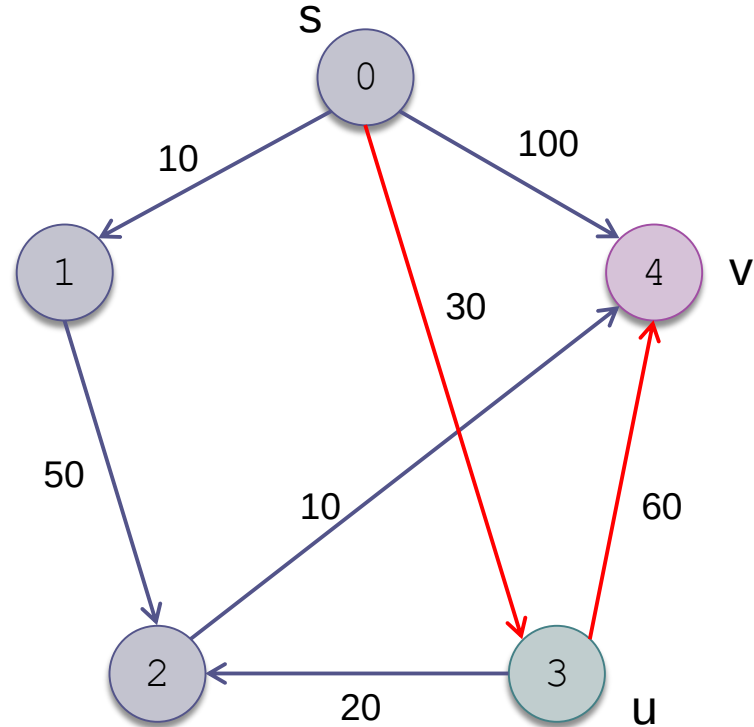
Dijkstra's Algorithm (cont.)

$S = \{0, 1\}$

$V - S = \{2, 3, 4\}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	100	0



$90 < 100$

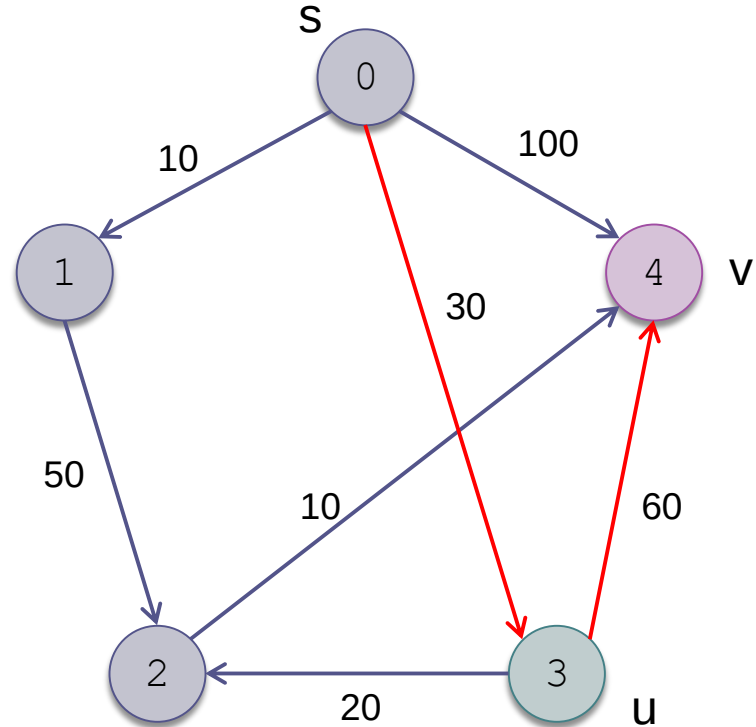
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1 \}$

$V-S = \{ 2, 3, 4 \}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	90	3



$90 < 100$

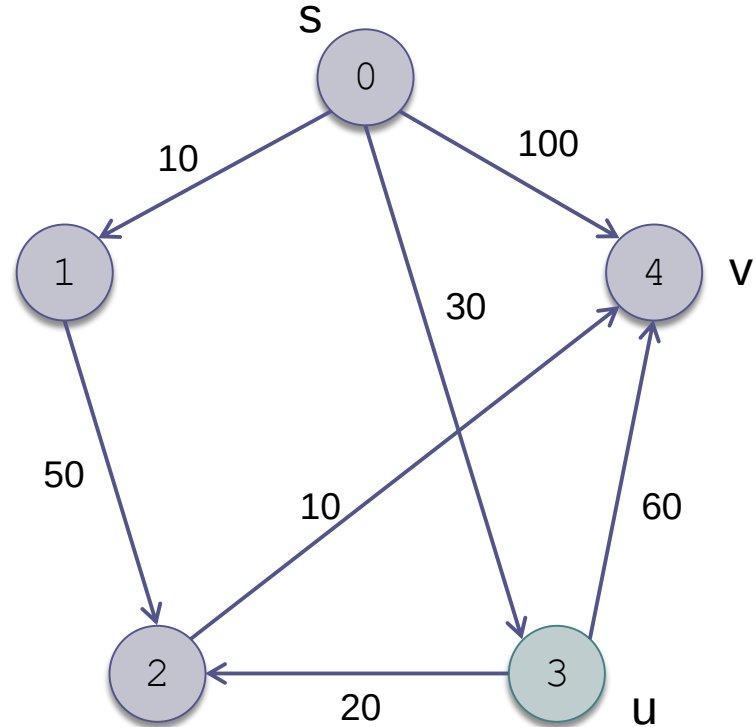
Dijkstra's Algorithm (cont.)

$S = \{0, 1\}$

$V-S = \{2, 3, 4\}$

$u = 3$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	90	3



Move u from $V-S$ to S

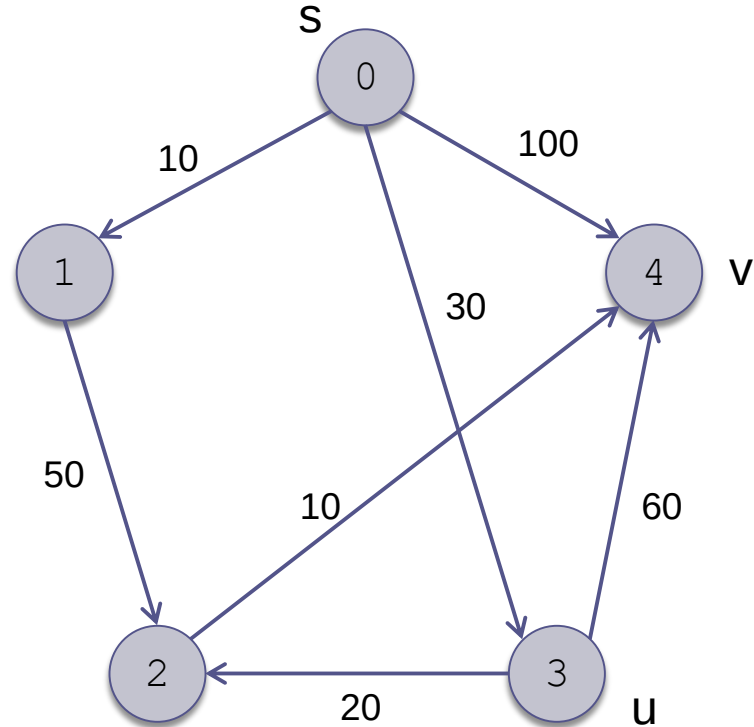
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3 \}$

$V-S = \{ 2, 4 \}$

$u = 3$

v	$d[v]$	$p[v]$
1	10	0
2	50	3
3	30	0
4	90	3



Move u from $V-S$ to S

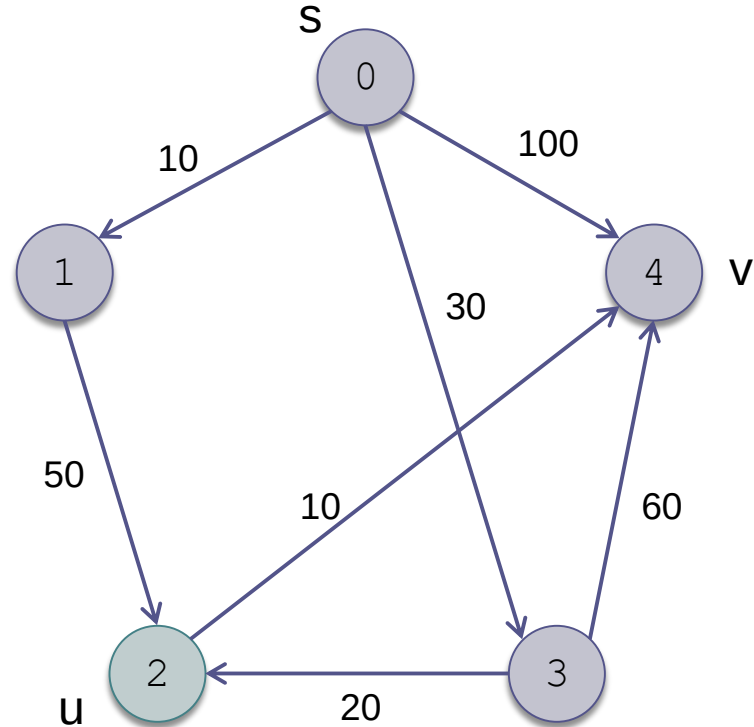
Dijkstra's Algorithm (cont.)

$S = \{0, 1, 3\}$

$V-S = \{2, 4\}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	90	3



Select vertex 2 from V-S

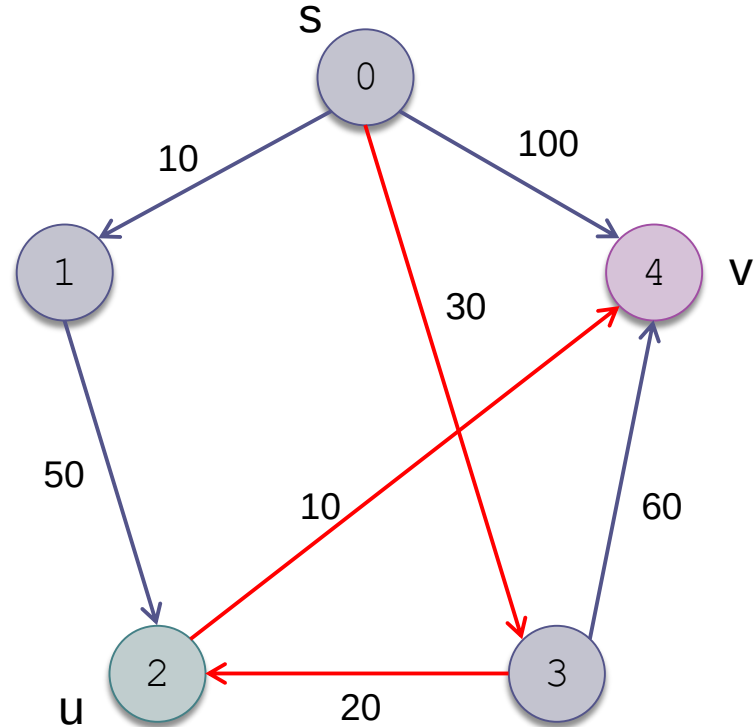
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3 \}$

$V - S = \{ 2, 4 \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	90	3



$$d[2] + w(2,4) = 50 + 10 = 60$$

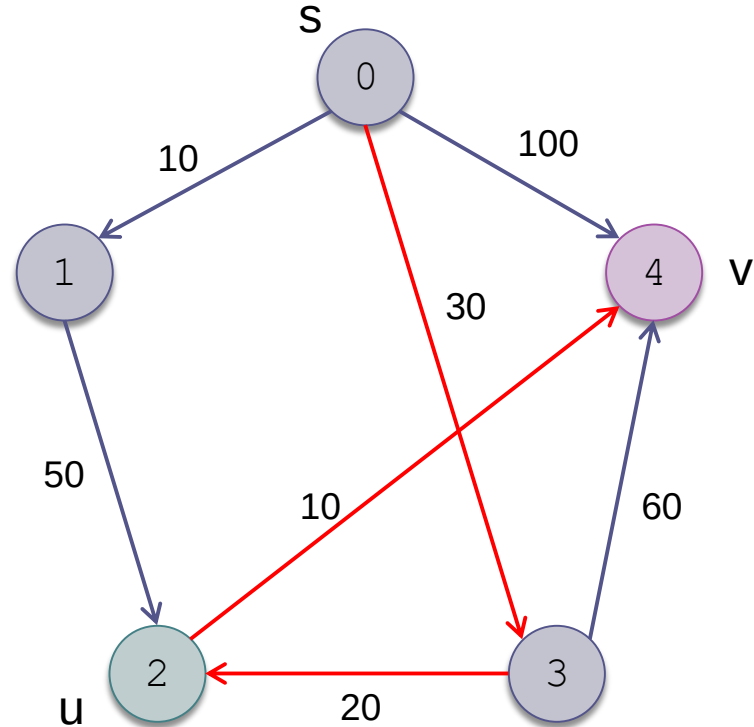
Dijkstra's Algorithm (cont.)

$S = \{0, 1, 3\}$

$V - S = \{2, 4\}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	90	3



$60 < 90$ ($d[4]$)

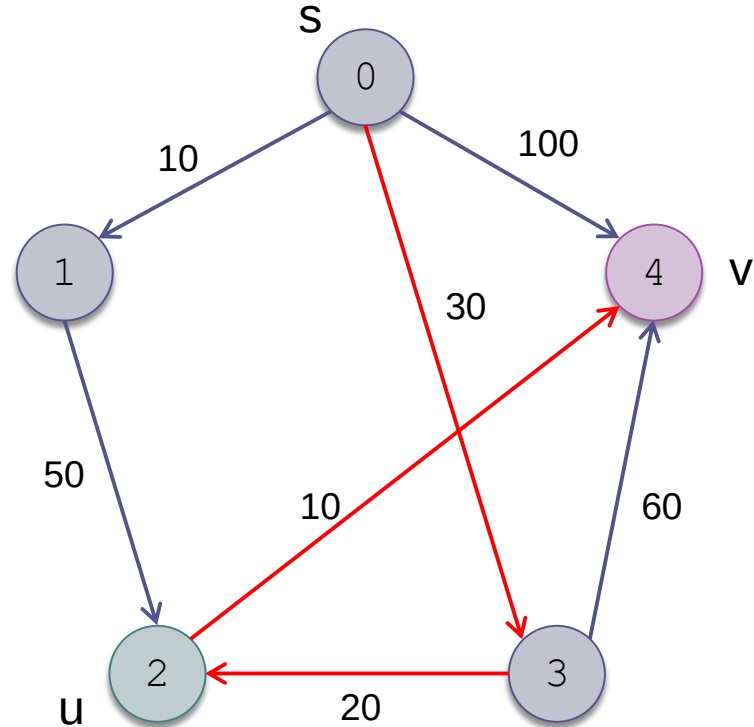
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3 \}$

$V-S = \{ 2, 4 \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	90	3



update $d[4]$ to 60 and $p[4]$ to 2

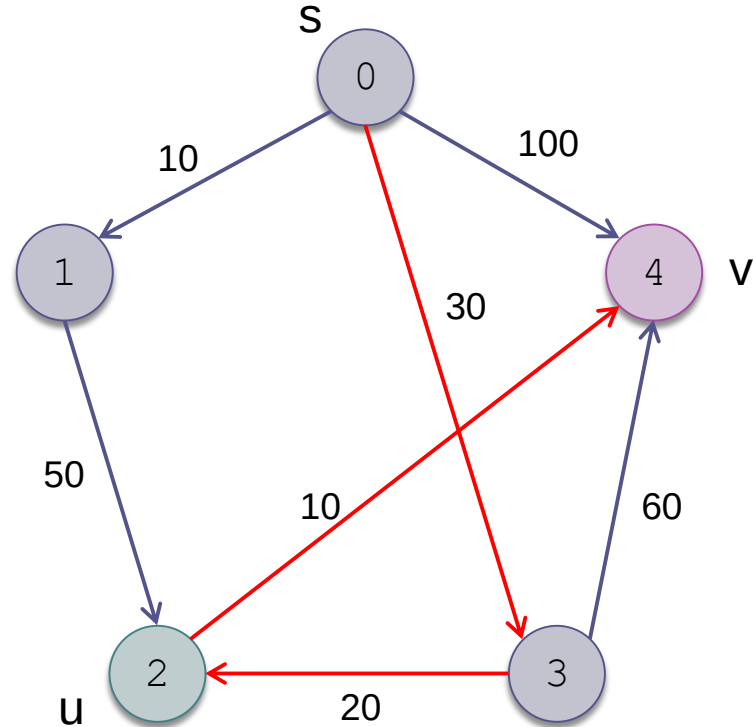
Dijkstra's Algorithm (cont.)

$S = \{0, 1, 3\}$

$V - S = \{2, 4\}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



update $d[4]$ to 60 and $p[4]$ to 2

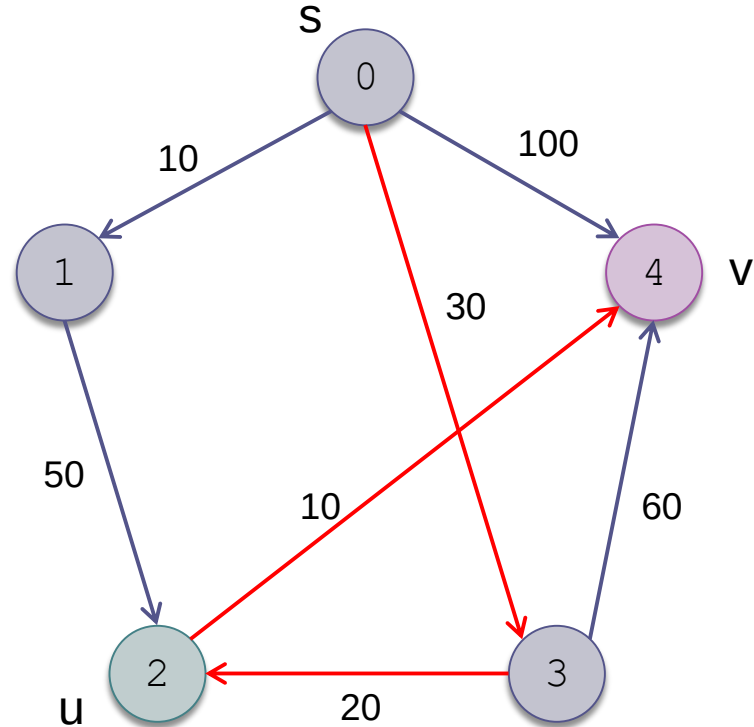
Dijkstra's Algorithm (cont.)

$S = \{0, 1, 3\}$

$V-S = \{2, 4\}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



Remove 2 from V-S

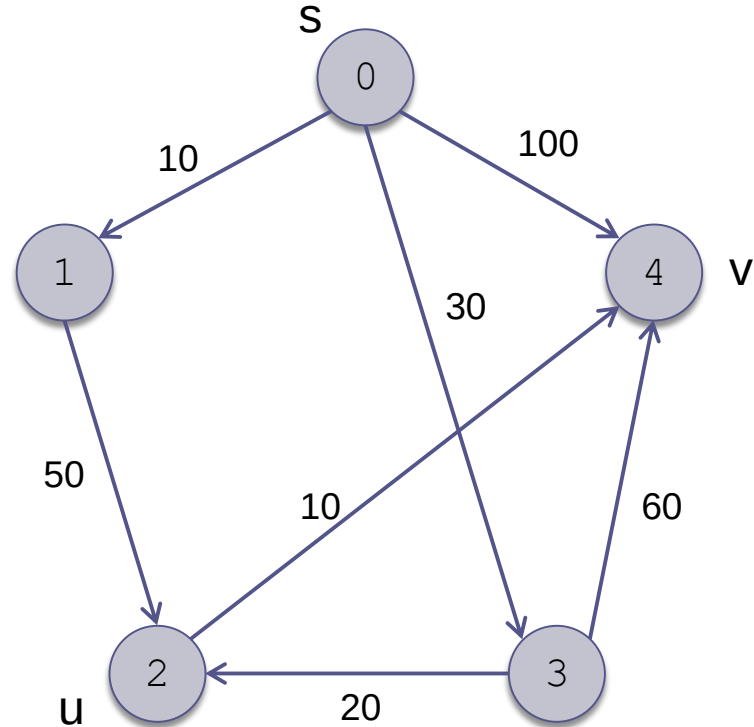
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3, 2 \}$

$V-S = \{ 4 \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



Remove 2 from V-S

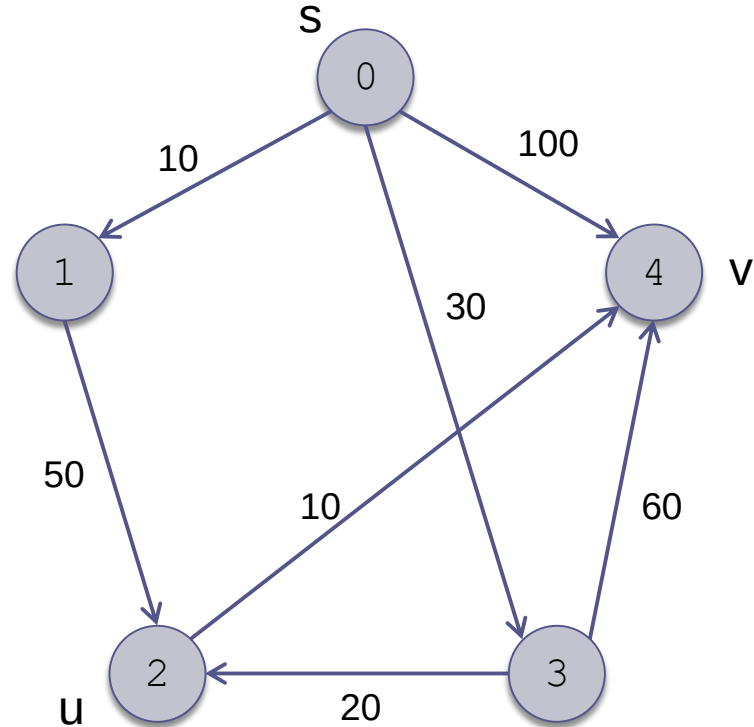
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3, 2 \}$

$V-S = \{ 4 \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



The final vertex in $V-S$ is 4

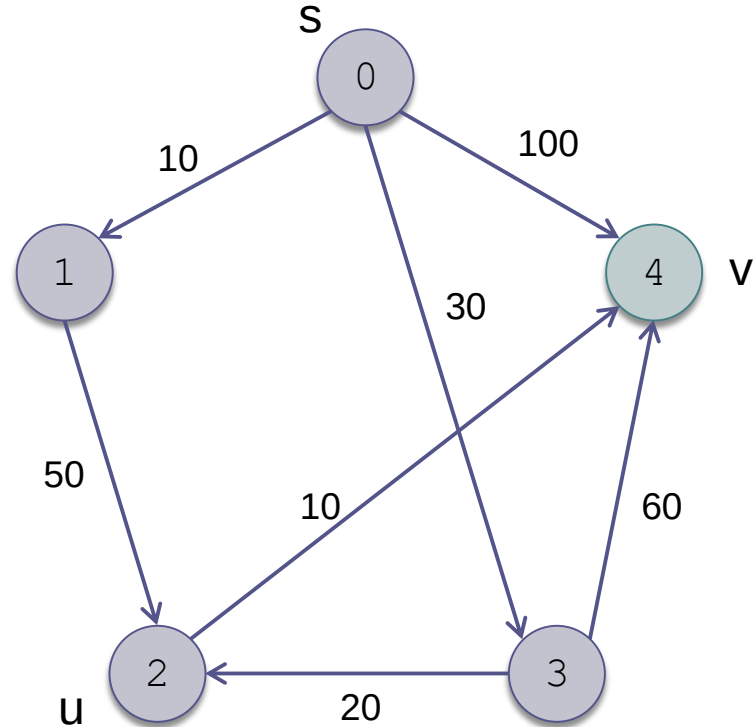
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3, 2 \}$

$V-S = \{ 4 \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



The final vertex in $V-S$ is 4

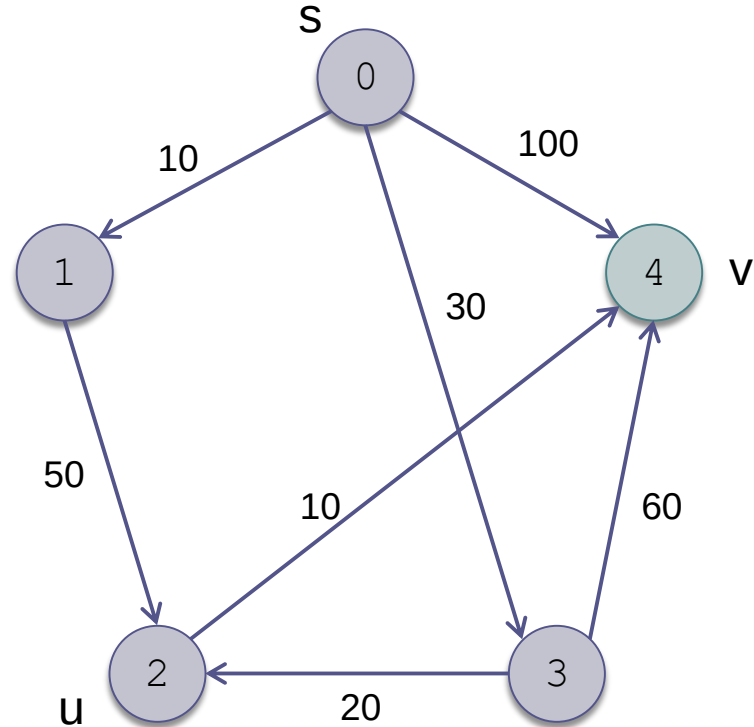
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3, 2 \}$

$V - S = \{ 4 \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



4 has no adjacent vertices; we move
4 into S

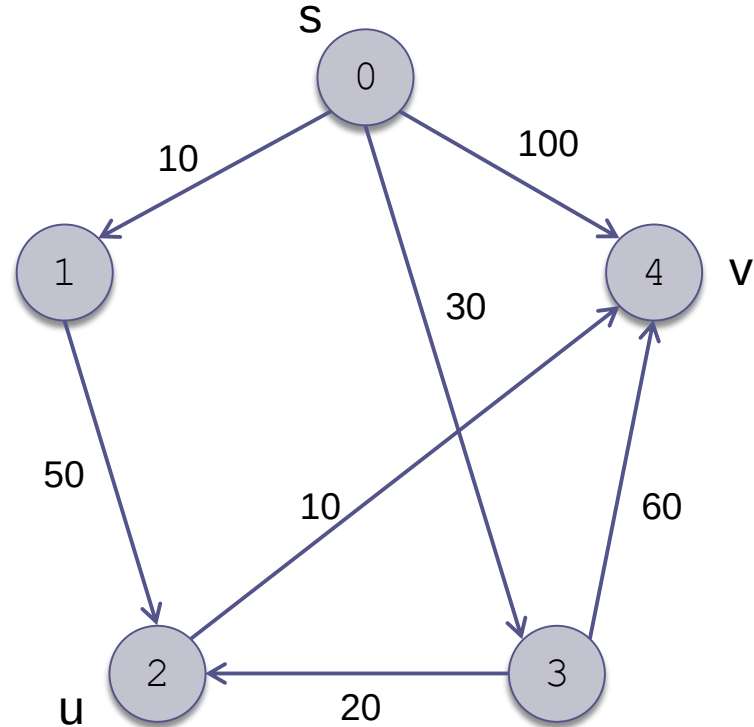
Dijkstra's Algorithm (cont.)

$S = \{0, 1, 3, 2, 4\}$

$V - S = \{\}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



4 has no adjacent vertices; we move
4 into S

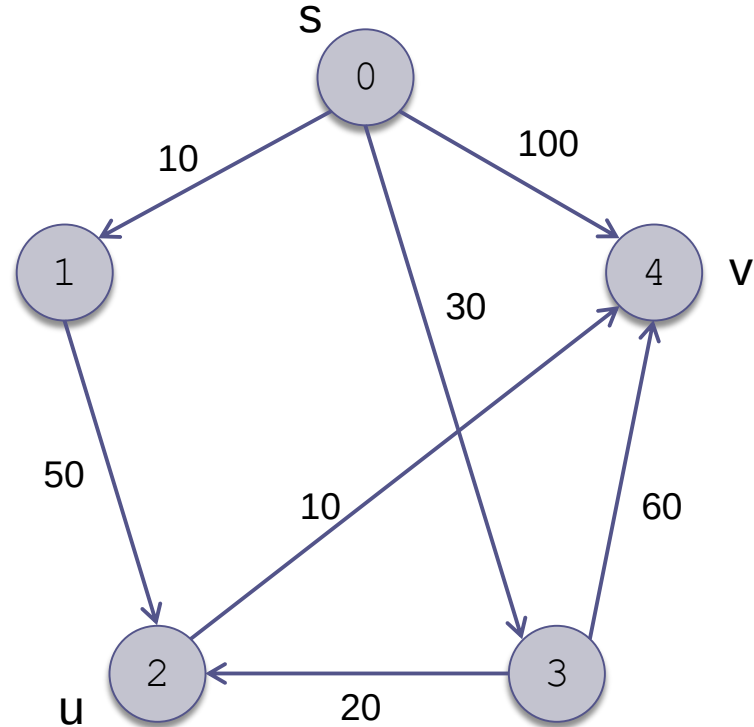
Dijkstra's Algorithm (cont.)

$S = \{ 0, 1, 3, 2, 4 \}$

$V - S = \{ \}$

$u = 2$

v	d[v]	p[v]
1	10	0
2	50	3
3	30	0
4	60	2



We are finished

Jeffrey J. McConnell

Analysis *of* Algorithms

Second Edition
An Active Learning Approach

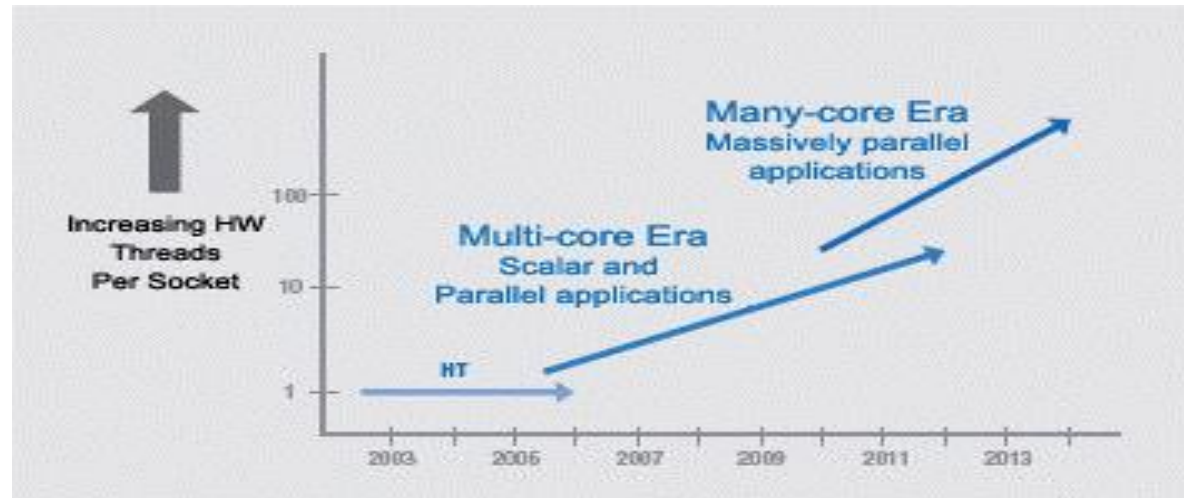


Chapter 9

Parallel Algorithms

Parallelism everywhere

- Warehouse-scale computers, data-centers, clouds
- Supercomputer
- GPUs
- Many-cores



Basic knowledge of parallel algorithms:

- Necessary for these industry platforms

Top500.org's November 2017 ranking

Position	Name	Country	Teraflops	Power (kW)
1	Sunway TaihuLight	China	93,015	15,371
2	Tianhe-2	China	33,863	17,808
3	Piz Daint	Switzerland	19,590	2,272
4	Gyokou	Japan	19,136	1,350
5	Titan	United States	17,590	8,209
6	Sequoia	United States	17,173	7,890
7	Trinity	United States	14,137	3,844
8	Cori	United States	14,015	3,939
9	Oakforest-PACS	Japan	13,555	2,719
10	K Computer	Japan	10,510	12,660

2019??

- <https://www.top500.org/statistics/list/53/countries/>

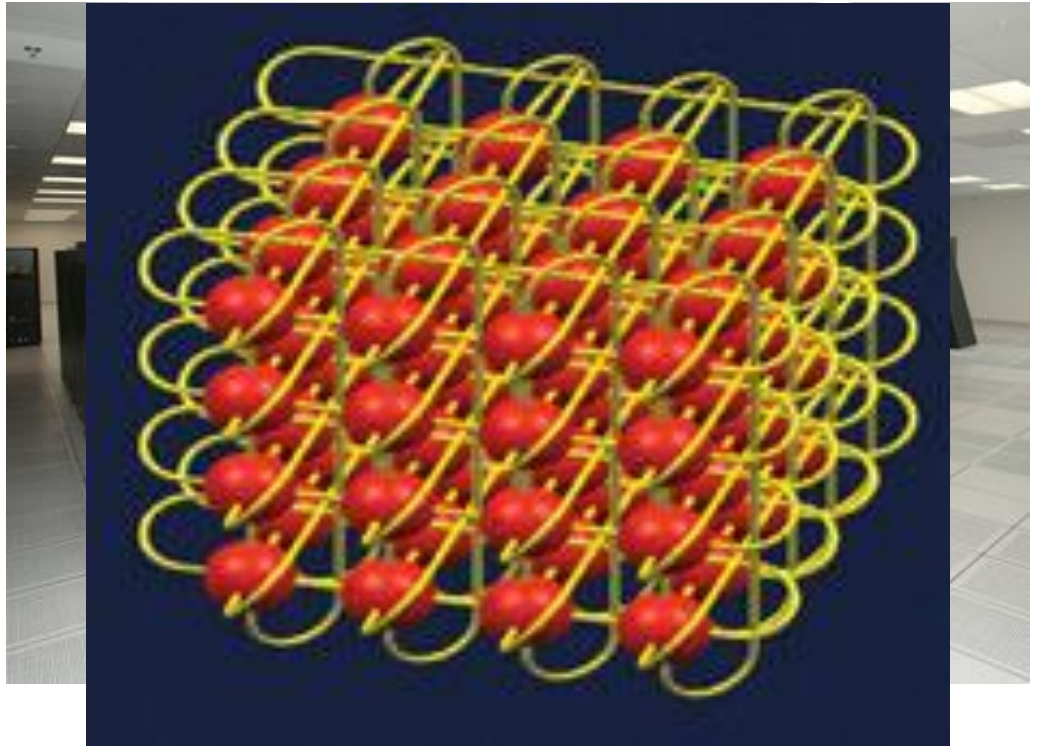
Some Real Systems: IBM BlueGene/L

212,992 CPUs

478 Tflops

At Lawrence
Livermore Nat'l Lab

≈ \$200 Million



3-d toroidal interconnect

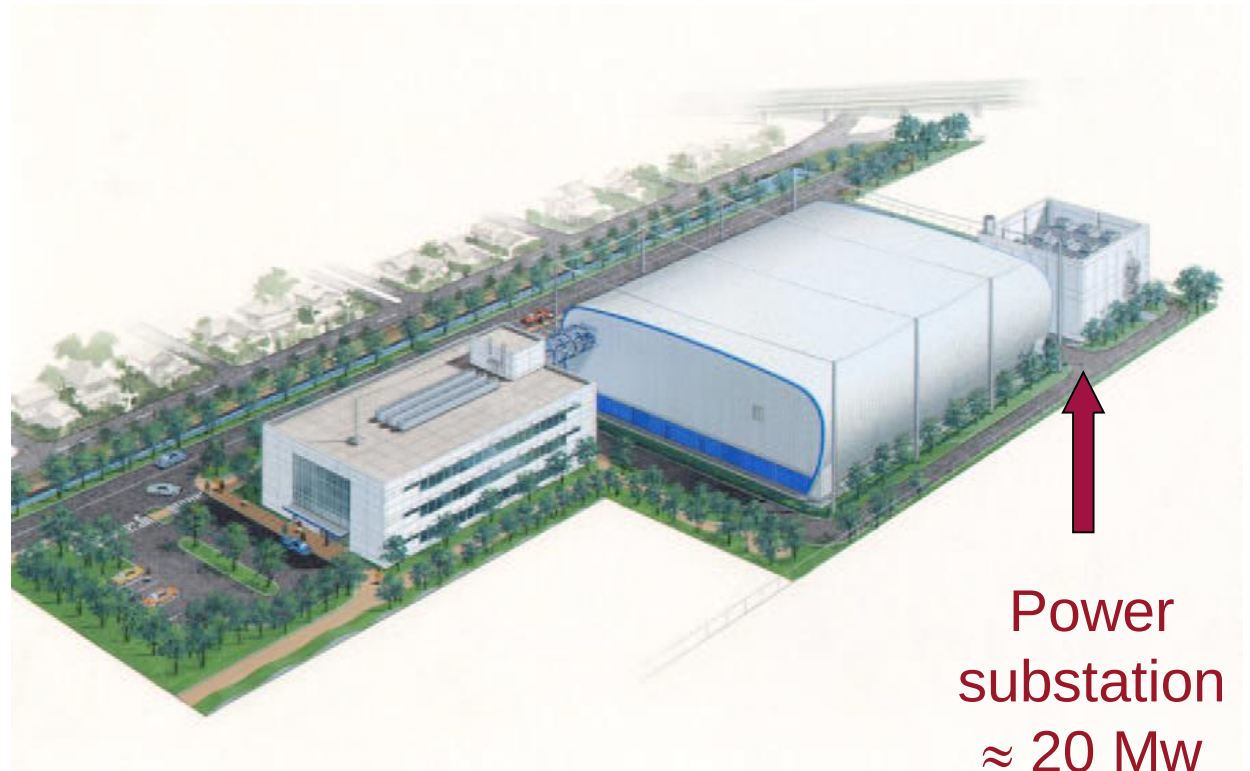
NEC Earth Simulator

5120 Processors

41 Tflops

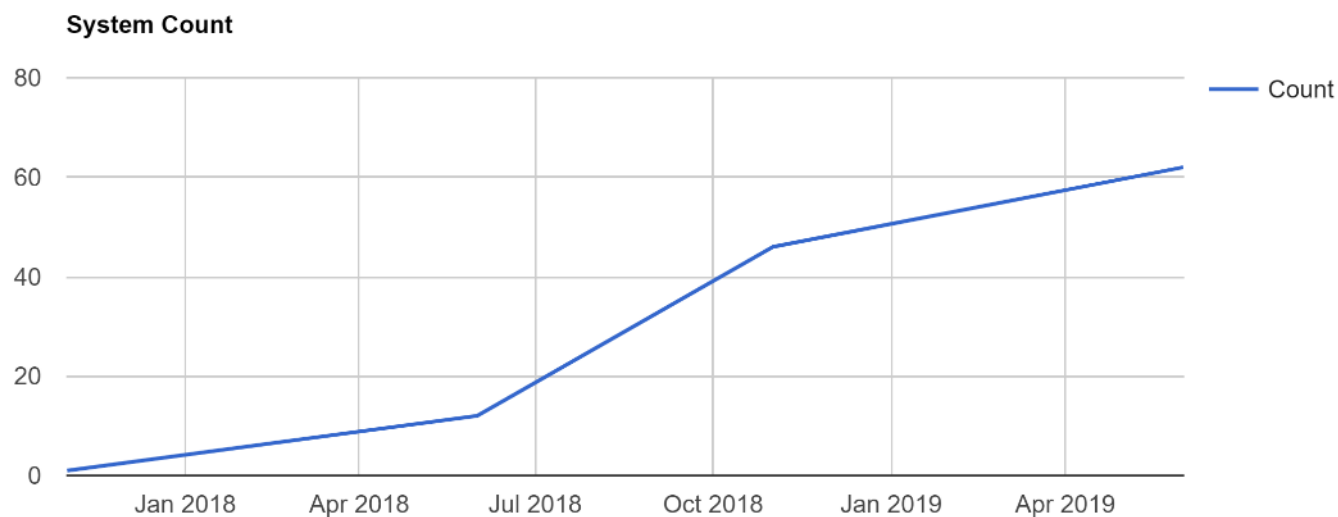
In Yokohama,
Japan

> \$500 Million



Power
substation
≈ 20 Mw

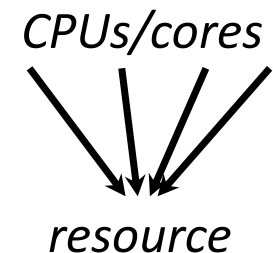
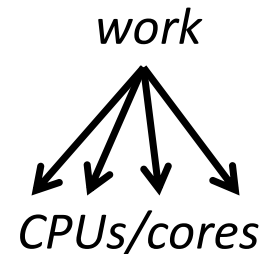
ACCELERATOR/CP FAMILY / NVIDIA VOLTA



List	Count	System Share (%)	Rmax (GFlops)	Rpeak (GFlops)	Cores
Jun 2019	62	12.4	406,316,070	621,405,705	7,481,412
Nov 2018	46	9.2	335,748,290	515,000,540	6,225,940
Jun 2018	12	2.4	224,559,400	360,593,742	4,488,720
Nov 2017	1	0.2	1,070,000	1,819,752	22,440

Parallel vs. concurrent

- **Parallel:** Using multiple processing resources (CPUs, cores) at once to solve a problem faster.
 - Example: A sorting algorithm that has several threads each sort part of the array.
- **Concurrent:** Multiple execution flows (e.g. threads) accessing a shared resource at the same time.
 - Example: Many threads trying to make changes to the same data structure (a global list, map, etc.).
- Many programmers confuse these two concepts.
 - Threads are often used to implement both.



Principles for Parallelism Analysis

- Algorithm speed
 - A count of the operations like in Chapters 2 through 8
- Speed-up
 - How much faster a parallel algorithm is
- Cost
 - The speed of an algorithm multiplied by the number of processors needed
 - For Chapters 2 to 8 the cost and the speed are the same because we used one processor

Speedup in Simplest Terms

$$\text{Speedup} = \frac{\text{Sequential execution time}}{\text{Parallel execution time}}$$

- Speed up range?

$$\text{Speedup} = \frac{\text{Sequential execution time}}{\text{Parallel execution time}}$$

- Best case speed up for M processors

$$\text{Speedup} = \frac{\text{Sequential execution time}}{\text{Parallel execution time}}$$

Efficiency

$$\text{Efficiency} = \frac{\text{Sequential running time}}{\text{Processors} \times \text{Parallel running time}}$$

$$\text{Efficiency} = \frac{\text{Speedup}}{\text{Processors}}$$

$$\text{Efficiency} = \frac{\text{Sequential running time}}{\text{Cost}}$$

Efficiency denoted in Quinn by $\varepsilon(n, p)$ for a problem of size n on p processors

- Examples:
- $T_s = 100$
- $T_p = 50$
- $P = 3$

- What is speed up?

- Examples:
 - $T_s = 100$
 - $T_p = 50$
 - $P = 3$
-
- What is speed up?
 - Efficiency?

Amdahl's Law

Let f be the fraction of operations in a computation that must be performed sequentially, where $0 \leq f \leq 1$. The maximum speedup ψ achievable by a parallel computer with n processors is

$$\psi \equiv S(p) \leq \frac{1}{f + (1-f)/n} \leq \frac{1}{f}$$

- The word “**law**” is often used by computer scientists when it is an observed phenomena (e.g, Moore's Law) and not a theorem that has been proven in a strict sense.
- However, Amdahl's law can be proved for traditional problems

Proof for Traditional Problems: If the fraction of the computation that cannot be divided into concurrent tasks is f , and no overhead incurs when the computation is divided into concurrent parts, the time to perform the computation with n processors is given by $t_p \geq ft_s + [(1 - f)t_s] / n$, as shown below:

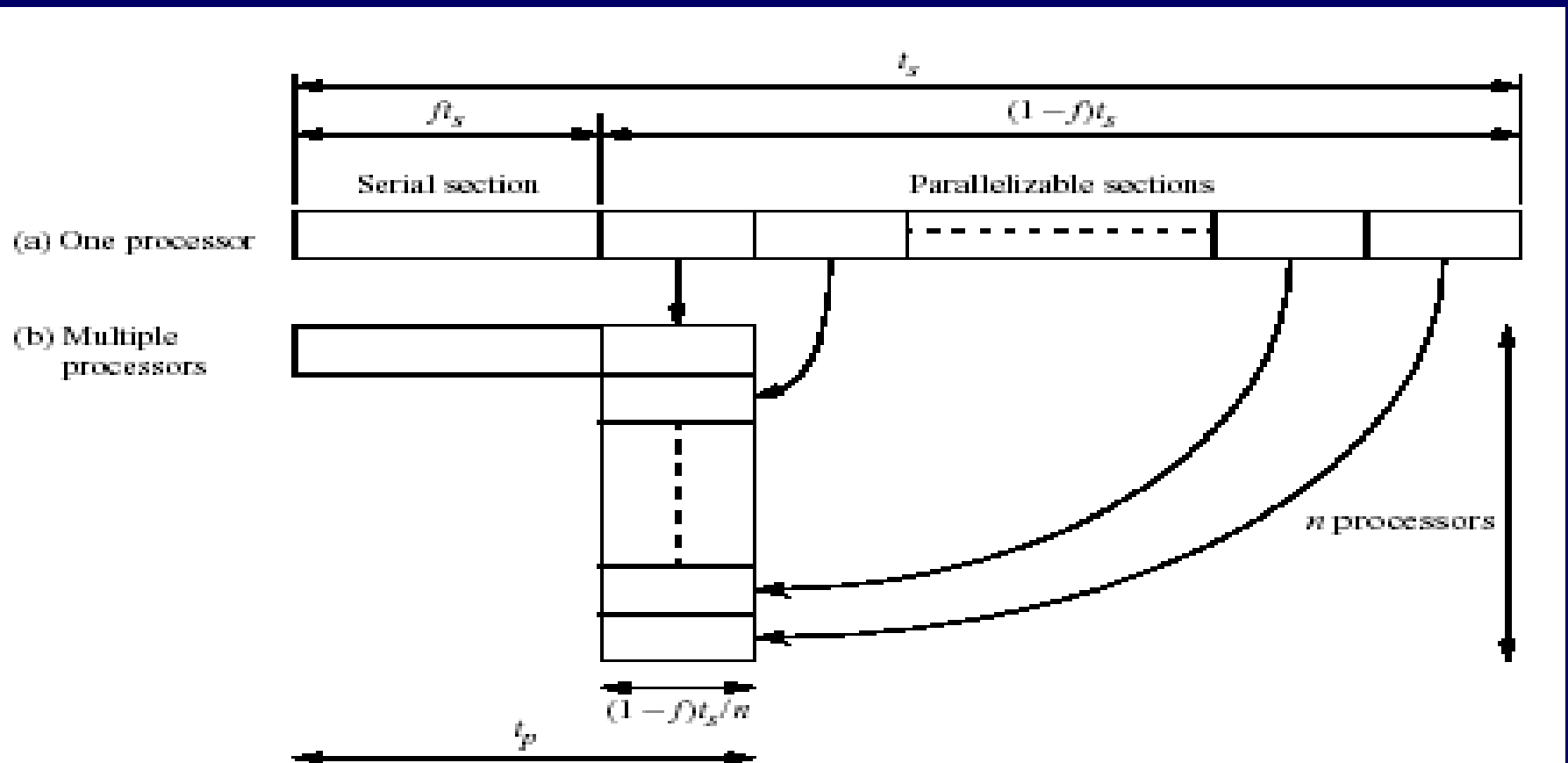


Figure 1.29 Parallelizing sequential problem — Amdahl's law.

Amdahl's Law

- Let $f = 0.1$
- $P = 20$ processors
- What is the upper bound for Speed up?

Amdahl's Law

- Let $f = 0.1$
- $P = 20$ processors
- $S \leq 1 / (0.1 + 0.9/20) = 6.9 \sim 7$

Amdahl's Law

- Let $f = 0.1$
- $P = 100$ processors
- What is the upper bound for Speed up?

Amdahl's Law

- Let $f = 0.1$
- $P = 100$ processors
- $S \leq 1 / (0.1 + 0.9/100) \sim 9$

Amdahl's Law

- Let $f = 0.1$
- $P = 1000$ processors ??
- Limits ?

Amdahl's Law

- Limits:
- $F = 0$, $f = 1$?
- $P = 1$ $P = 100000$
- Limits ?

Computer System Categories

- Single Instruction Single Data (SISD) (PC)
- Single Instruction Multiple Data (SIMD)
- Multiple Instruction Single Data (MISD) (Pipeline)
- Multiple Instruction Multiple Data (MIMD) (MultiProcessors)



Processor Architectures

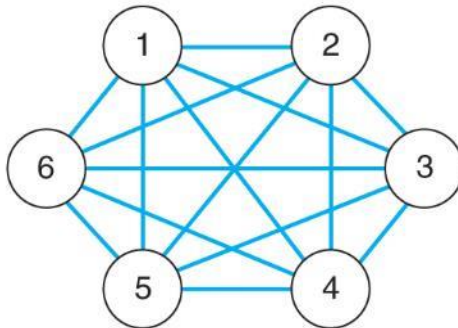
- Loosely coupled machines
 - Each computer has its own memory
- Tightly coupled machines
 - Computers share a centralized memory
- Processor communication
 - Over network cables
 - By storing information in particular memory cells

Network configurations

- Fully connected
- Linear
- Ring
- Two-dimensional mesh

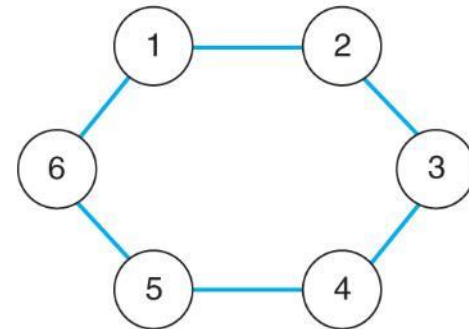
■ FIGURE 9.1

A fully connected and a linear network configuration



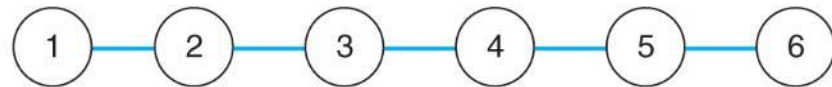
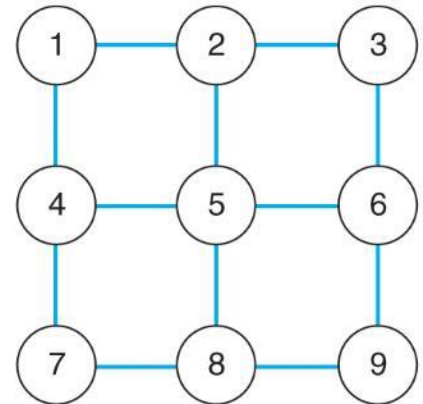
■ FIGURE 9.2

A ring network configuration



■ FIGURE 9.3

A mesh network



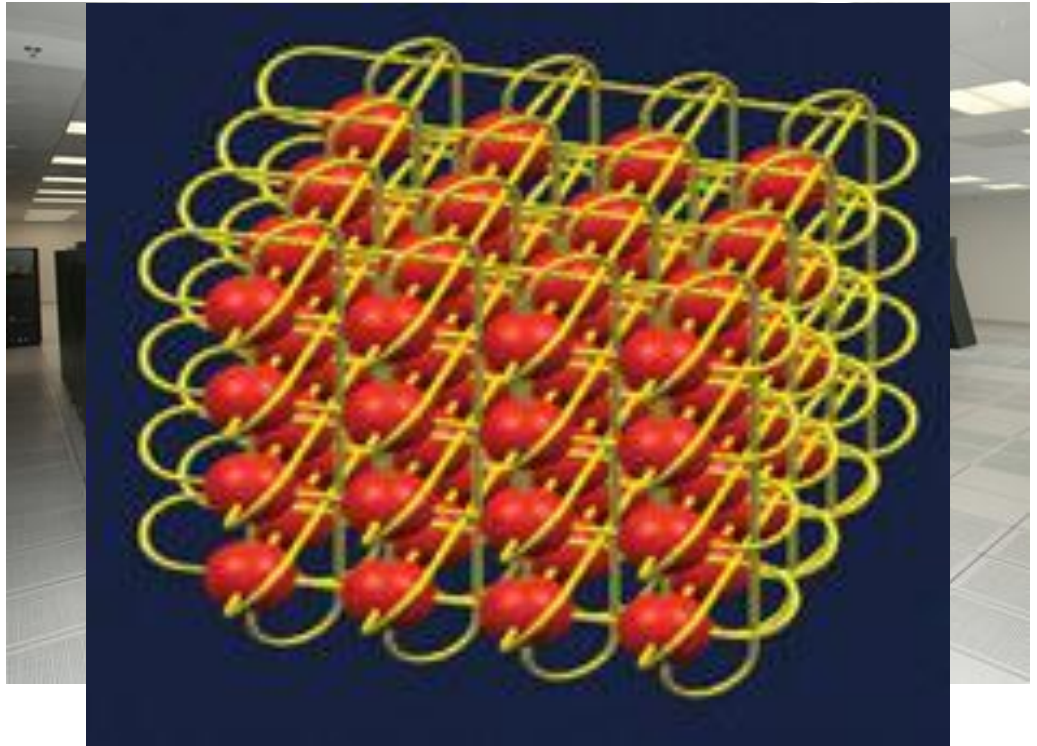
Some Real Systems: IBM BlueGene/L

212,992 CPUs

478 Tflops

At Lawrence
Livermore Nat'l Lab

≈ \$200 Million



3-d toroidal interconnect

The PRAM Model

- Parallel Random Access Machine
 - Tightly coupled machine
 - Each processor has a small number of registers for temporary storage
 - All processors run on the same cycle:
 - All read data from memory in step 1
 - All process data in step 2
 - All write data to memory in step 3



Memory Contention

- If multiple computers try to write to the same memory location, we can have erratic results
- We can have exclusive reads, where only one processor can read from one location, while other processors can read from other locations

Memory Contention

- We can have concurrent reads, where more than one processor can read from the same memory location
- We can have exclusive writes, where only one processor can write to one location, while other processors can write to other locations – multiple writes to a location is an error



Memory Contention

- We can have concurrent writes, where more than one processor can write to the same memory location

Write Conflict Resolution

- In concurrent writes, the system will use some mechanism to determine which succeeds:
 - Priority
 - Highest priority processor succeeds
 - Arbitrary
 - Random processor succeeds
 - Common
 - All values being written must be the same
 - Combining
 - Values combined by a predefined operator



Four Types of Memory Access

- Concurrent Read Concurrent Write (CRCW)
- Concurrent Read Exclusive Write (CREW)
- Exclusive Read Concurrent Write (ERCW)
- Exclusive Read Exclusive Write (EREW)



Broadcasting Data in a CREW PRAM Model

- Transfer a data value to all of the other processors
- This process will take two cycles

Broadcasting Data in a CREW PRAM Model Algorithm

```
P1 writes the data value into M1  
Parallel Start  
  for k = 2 to p do  
    Pk reads the data value from M1  
  end for  
Parallel End
```